Nova: A Glass-Box Compiler with Verified Bytecode and an Interactive Studio for Compiler Education

[Author Name 1]
[Department Name]
[Institution Name]
[City], [Country]
[email address]

[Author Name 2]
[Department Name]
[Institution Name]
[City], [Country]
[email address]

[Author Name 3]
[Department Name]
[Institution Name]
[City], [Country]
[email address]

***Abstract*—Production compilers are black boxes: the automata, parse tables, intermediate representations and data-flow facts that they compute are discarded or buried in textual dumps, which makes compiler construction difficult to learn and compiler output difficult to trust. This paper presents Nova, a complete optimising compiler for an original imperative language, written from first principles in 7,059 lines of dependency-free Python, that retains and exposes every artefact it computes. Nova builds its scanner from regular expressions by the direct (followpos) method, parses with a recursive-descent parser that is cross-checked against SLR(1), canonical LR(1) and LALR(1) tables built from the same 58-production grammar, type-checks with an L-attributed definition, emits backpatched three-address code, applies eight optimisation passes driven by four data-flow analyses, allocates registers from next-use information, constructs static single assignment (SSA) form and classifies loops for parallelism. Its bytecode is proved safe by an abstract-interpretation verifier before it runs in a resource-bounded virtual machine, and every program is executed by three independent back-ends whose outputs must agree. A local web application, Nova Studio, recompiles as the user types and renders every phase as an interactive instrument: a scanner trace through the DFA, a browser for all 116 states of the LALR(1) automaton and its table, a parse player, drawn syntax trees, DAGs and dominator trees, data-flow facts on the flow graph, a register-occupancy timeline and a virtual-machine stepper. On an eleven-program benchmark suite, LALR(1) matches SLR(1)'s 116 states with 75% fewer table entries than canonical LR(1); every data-flow analysis converges within the number of passes that theory predicts; the optimiser shrinks bytecode by up to 23% on non-trivial programs without changing any output; all eight hostile bytecode programs are stopped; and ten of the eleven programs compile in under 40 ms on a battery-throttled laptop. A scaling study shows where the glass box costs: the bookkeeping of modules 5–7 grows between quadratically and cubically with program size. Exposing every phase also surfaced six latent defects, including missing ϕ-functions in functions that begin with a loop, all now fixed.**

***Index Terms*—compiler construction, LALR parsing, data-flow analysis, static single assignment, register allocation, bytecode verification, abstract interpretation, sandboxing, program visualisation, algorithm animation, computing education**

I. Introduction

A compiler is the classic example of a large program assembled from a pipeline of well-understood algorithms [1]–[3]. Each stage—lexical, syntactic and semantic analysis, intermediate-code generation, optimisation and code generation—has a precise textbook formulation, yet in practice the intermediate results are invisible. Production compilers discard their token automata and parse tables once they have been generated, keep their intermediate representations inside the process, and expose them, if at all, through voluminous textual dumps [4]. Scanner and parser generators [5]–[7] hide the constructions they perform behind a specification language. Students therefore learn that LALR(1) tables are "as small as SLR(1) tables but almost as powerful as canonical LR(1)" as an assertion rather than as a measurement; they see the fixed point of a data-flow analysis described but never watch it being reached; and they are told that a stack machine pushes an activation record on every call without ever seeing one.

Opacity also undermines trust. Compilers are themselves defective—random differential testing found hundreds of bugs in mature C compilers [8]—and systems that execute untrusted code do not trust the code generator at all: the Java Virtual Machine (JVM), WebAssembly engines and the Linux eBPF subsystem all verify code before running it [9]–[12]. A compiler intended for teaching should demonstrate both ideas, not merely describe them.

This paper describes *Nova*, a *glass-box* compiler designed around three principles: (i) every artefact that a phase computes is retained and can be inspected; (ii) every non-trivial claim is checked by an independent mechanism—two parsers, three executable back-ends and a bytecode verifier; and (iii) every number is measured by the compiler itself rather than asserted. Fig. 1 shows Nova Studio, the interactive front end, compiling the recursive Fibonacci benchmark: the pipeline rail reports the status and cost of each of the nine stages, the editor highlights the source, and the inspector summarises the compilation and the output of three independent back-ends. Algorithm-animation research has long argued that learners benefit most when they *engage* with a visualisation rather than merely view it [13], [14]; Nova Studio is therefore built from instruments that respond—a token can be clicked to trace it through the automaton, a parse can be scrubbed action

by action, a virtual machine can be stepped.

The contributions of this work are:

- A complete compiler for an original language, implemented in 29 Python modules (7,059 non-blank lines) with no third-party dependencies, covering scanning, parsing, semantic analysis, intermediate code, optimisation, register allocation, SSA construction, dependence analysis, bytecode verification and sandboxed execution (Section III).
- A side-by-side construction of SLR(1), canonical LR(1) (CLR) and LALR(1) tables for the full language grammar, with a bottom-up parse that independently validates the recursive-descent parser, and a browser for every state and table entry of the LALR(1) automaton (Sections III-E and IV-B).
- A safety pipeline in which an abstract-interpretation verifier proves six properties of all generated bytecode before a resource-bounded virtual machine (VM) executes it, evaluated against eight hand-written hostile programs (Sections III-L and IV-H).
- A differential-validation discipline in which unoptimised bytecode, optimised bytecode and separately generated register-machine code must produce identical output for every program (Section IV-F).
- Nova Studio, a dependency-free local web application that recompiles as the user types and renders every phase as an interactive instrument, together with a security model for a localhost development server (Section III-M).
- A reproducible evaluation on eleven benchmark programs and a synthetic scaling series: scripts regenerate every number, table and screenshot in this report, and the evaluation itself exposed six latent defects, which we analyse (Section IV).

The rest of the paper is organised as follows. Section II reviews the literature that Nova builds on and identifies the gap it addresses. Section III presents the methodology: the language, the architecture, each phase, the studio and the evaluation protocol. Section IV reports and discusses the results, and Section V concludes and outlines future work.

II. Literature Review

A. Foundations of Compiler Construction

The classical organisation of a compiler, and most of the algorithms used in Nova, are presented in the standard texts of Aho *et al.* [1], Appel and Palsberg [2], Cooper and Torczon [3] and Muchnick [15]. These texts describe precisely the artefacts that Nova exposes—automata, parse tables, attribute flows, flow graphs and data-flow solutions—but, being books, they present them statically and for small hand-worked examples.

B. Lexical Analysis

Thompson's construction translates a regular expression into a nondeterministic finite automaton (NFA) that is then determinised [16]. McNaughton and Yamada [17] and Glushkov [18] instead built an automaton directly from the *positions* of the symbols in an expression; Berry and Sethi [19] derived the same position automaton from expression derivatives, and Aho *et al.* present it as the *direct method* based on the functions nullable, firstpos, lastpos and followpos [1]. Moore's partition refinement [20] and Hopcroft's $O(n \log n)$ algorithm [21] minimise the resulting deterministic finite automaton (DFA). Lex [5] packages these constructions behind a specification language.

C. Syntax Analysis

Knuth introduced LR(k) parsing [22]. DeRemer's SLR(k) [23] and LALR(k) [24] variants made it practical by keeping the LR(0) automaton and refining only the look-aheads, and DeRemer and Pennello [25] showed how to compute LALR(1) look-aheads efficiently without building the canonical LR(1) collection. Yacc [6] and its successor Bison generate LALR(1) tables and can describe the automaton in a textual report [26]; ANTLR [7] generates top-down predictive parsers. Hand-written recursive-descent parsers with panic-mode recovery remain common in production front ends because they give precise control over diagnostics [1].

D. Semantic Analysis

Knuth's attribute grammars [27] formalised syntax-directed translation with inherited and synthesised attributes. Lewis, Rosenkrantz and Stearns [28] characterised the *L-attributed* subclass, whose attributes can be evaluated in a single left-to-right traversal—the property that lets Nova's type checker run in one pass.

E. Optimisation and Data-Flow Analysis

Allen formalised control-flow analysis over basic blocks, dominators and natural loops [29]. Kildall unified global optimisations as iterative, lattice-based data-flow analyses [30], and Kam and Ullman established when such iteration converges and how quickly [31], [32]. Cocke [33] and Alpern, Wegman and Zadeck [34] developed common-subexpression elimination and value numbering; strength reduction [35] and peephole optimisation [36] complete the classical repertoire. Cytron *et al.* [37] showed how to construct static single assignment (SSA) form efficiently by placing ϕ-functions on iterated dominance frontiers, building on fast dominator algorithms [38], [39]; SSA in turn simplifies sparse analyses such as conditional constant propagation [40].

F. Code Generation and Register Allocation

Register allocation by graph colouring [41] and by linear scan [42] dominates production compilers. The simple code generator taught in compiler courses instead allocates registers within a basic block using *next-use* information [1], spilling the value whose next use is furthest away—Belady's optimal replacement rule [43] applied to registers. LLVM [44] popularised a typed, SSA-based intermediate representation that front ends can emit as plain text.

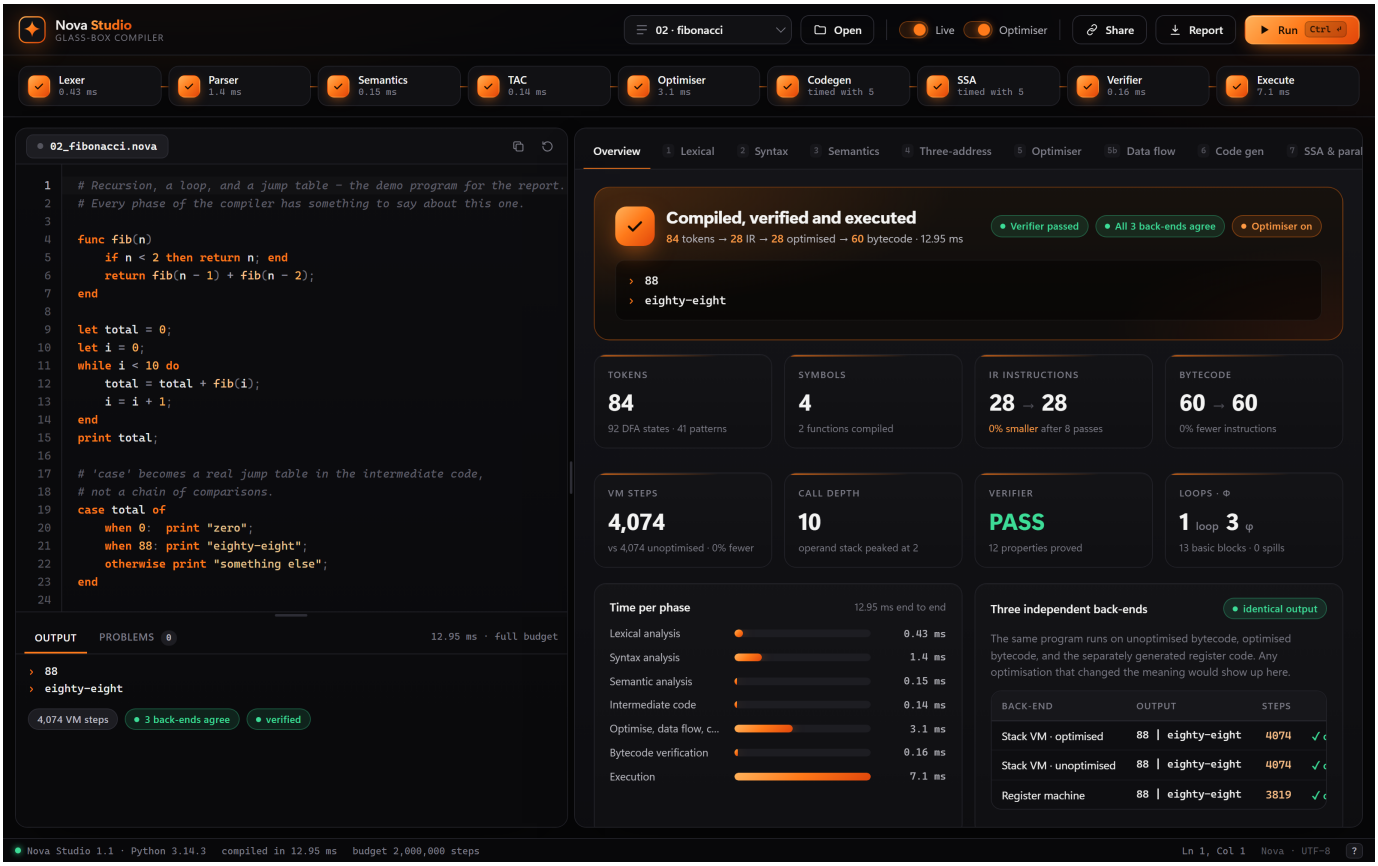


Fig. 1. Nova Studio compiling `02_fibonacci.nova`. Top: toolbar with example picker, live-compile and optimiser switches, sharing, report download and *Run*. Below it, the pipeline rail shows each of the nine stages with its status and its cost in this compilation. Left: the syntax-highlighted editor and the program output, with a summary of the three back-ends. Right: the inspector's overview of the compilation, including the hero summary, key measurements, per-phase timing and the agreement of the three back-ends. All panels are rendered from one `CompileResult`.

G. Dependence Analysis and Parallelisation

Allen and Kennedy [45], [46] and Wolfe [47] developed the dependence analysis that underpins vectorisation and loop restructuring; Kennedy and McKinley [48] used it to drive loop fusion and distribution. OpenMP [49] exposes the resulting parallel-loop and reduction patterns to programmers.

H. Code Verification and Sandboxing

Abstract interpretation [50] provides the theory for proving program properties by computing over abstract domains. The JVM applies it to bytecode: every class is verified for type and stack safety before execution [9], [10]. Proof-carrying code [51] and software fault isolation [52] are alternative routes to running untrusted code safely, and WebAssembly validation [11] and the Linux eBPF verifier [12] are their modern descendants. Nova's verifier is a small instance of the JVM approach, complemented by resource limits enforced at run time.

I. Compiler Correctness

CompCert [53] proves a compiler correct once and for all; translation validation [54] checks each individual compilation instead; differential testing [8], [55] compares the behaviour of independent implementations on the same input. Nova adopts the lightweight end of this spectrum, comparing three back-ends on every program it compiles.

J. Program Visualisation and Educational Compilers

A meta-study of algorithm visualisation found that the *form* of engagement, not the quality of the graphics, predicts learning: visualisations that learners control, predict and respond to outperform those they only watch [13]. Sorva *et al.* [14] reviewed program-visualisation systems for introductory programming and reached the same conclusion, and Guo's Online Python Tutor [56] showed that a stepping debugger that draws the run-time state is effective at scale. Graph-drawing research supplies the layouts such tools need: Reingold and Tilford's tidy tree layout [57] and Sugiyama's layered layout for directed graphs [58].

COOL [59] and ChocoPy [60] are languages designed for compiler courses: students implement phases against a reference compiler, which teaches construction but does not visualise the reference compiler's internals. JFLAP [61] animates automata and LL(1) and SLR(1) parsing for formal-language courses but stops before semantic analysis. Compiler Explorer [62] shows interactively the assembly and intermedi-

TABLE I
CAPABILITIES OF EXISTING TOOL FAMILIES AND NOVA

Capability	GCC/ LLVM	Bison/ ANTLR	JFLAP	CE[a]	Nova
Regex → automaton construction	○	○	●	○	●
Scanner trace for a chosen token	○	○	◐	○	●
LR table construction and comparison	○	◐	◐	○	●
LR automaton browser, parse replay	○	◐	◐	○	●
Annotated tree / attribute flow	◐	○	○	◐	●
IR and per-pass optimisation effects	◐	○	○	●	●
Data-flow fixed-point iteration	○	○	○	○	●
Register-allocation decisions	◐	○	○	◐	●
SSA, dominator tree, frontiers	◐	○	○	◐	●
Bytecode verification and sandbox	○	○	○	○	●
VM stepping with activation records	○	○	○	○	●
Independent cross-checking	○	○	○	◐	●
All phases of one program, live	○	○	○	◐	●
Offline, no third-party packages	●	●	◐	◐	●

● supported; ◐ partial, textual or limited; ○ not supported; based on each tool's documented features. [a]Compiler Explorer.

ate code that production compilers generate, but not the front-end artefacts or the analyses that justify each transformation.

K. Research Gap

Table I compares these tool families with Nova. To our knowledge, no existing tool lets a learner follow a single program from regular-expression automata through LR tables, attribute flow, data-flow fixed points, register allocation and SSA to verified, sandboxed execution in one live environment, while cross-checking each step and letting the learner drive every instrument. That gap motivates Nova.

III. METHODOLOGY

A. Research Approach and Design Principles

Nova was developed as design-science research: an artefact is built against explicit requirements and then evaluated empirically. The requirements come from the eight modules of the Compiler Design syllabus that the project addresses (Table II) and from five design principles:

1) *Glass box.* No phase discards its working. The scanner keeps its syntax tree, followpos table, both automata and, for every token, the DFA states it walked; the parsers keep their item sets, tables and traces; the optimiser records the code after every pass; each data-flow analysis records every iteration; the VM records its trace, including the locals of the current frame at every step. One `CompileResult` object holds all of it.
2) *One pipeline, many views.* The command-line interface, the desktop viewer, the HTML report and Nova Studio all render the same `CompileResult`, so they cannot disagree.

TABLE II
SYLLABUS MODULES AND THEIR IMPLEMENTATION

Module	Techniques implemented	Source modules
M1 Lexical	regex → DFA (direct method), Moore minimisation, maximal munch, per-token scanner trace; LLVM IR	`regexdfa`, `lexer`, `llvmir`
M2 Syntax	FIRST/FOLLOW, LL(1), left recursion, SLR/CLR/LALR, automaton browser, recursive descent, panic mode, parse replay	`grammar`, `lr`, `parser`
M3 Semantics	scoped symbol table, L-attributed SDD, type checking, drawn annotated tree	`semantic`, `ast_nodes`
M4 Intermediate	three-address code, backpatching, jumping code, jump tables	`tac`
M5 Optimisation	blocks, CFG, DAG/LVN, eight passes, four data-flow analyses drawn on the graph	`cfg`, `dag`, `dataflow`, `optimizer`
M6 Code gen.	next use, register allocation with a register timeline, activation records, two executable targets	`codegen`, `regvm`, `bytecode`, `vm`
M7 SSA, parallel	dominator tree, dominance frontiers, ϕ placement, dependence analysis	`ssa`, `parallel`
M8 Contemporary	bytecode verification, sandboxing, VM stepper, hostile code	`verifier`, `vm`, `attacks`

3) *Trust nothing that can be checked.* Two parsers accept the same token stream, three back-ends execute every program, and a verifier proves the bytecode safe before it runs.
4) *Measure, do not assert.* Every statistic in the user interface, and in this paper, is produced by the compiler or by the scripts in `report/tools`.
5) *Zero dependencies.* Only the Python standard library is used, so the tool runs on any machine with Python by double-clicking a launcher; the studio's page is plain HTML, CSS and JavaScript with no framework and no build step.

B. The Nova Language

Nova is a small imperative language whose syntax is deliberately not C-like (Fig. 2). A program is a sequence of function declarations and statements; statements are `let` declarations, assignments, `if–then–else–end`, `while–do–end`, `case–of–when–otherwise–end`, `print`, `return` and call statements. Expressions follow the usual precedence chain from `or` through `and`, equality, relational, additive and multiplicative operators to unary `-` and `not`. The types `int`, `bool` and `str` are inferred and checked; `+` also concatenates two strings. Functions take integer parameters and may be recursive, and scopes nest and shadow. The lexical specification has 41 token patterns (19 keywords, literals, identifiers,

```
func fib(n)
    if n < 2 then return n; end
    return fib(n - 1) + fib(n - 2);
end

let total = 0;
let i = 0;
while i < 10 do
    total = total + fib(i);
    i = i + 1;
end
print total;

case total of
    when 0:  print "zero";
    when 88: print "eighty-eight";
    otherwise print "something else";
end
```

Fig. 2. The Fibonacci benchmark (`02_fibonacci.nova`, comments removed): recursion, a loop and a `case` statement that compiles to a jump table. It prints `88` and `eighty-eight`.

operators, whitespace and comments), and the grammar has 58 productions over 39 terminals and 20 non-terminals.

C. System Architecture

Fig. 3 shows the architecture. The driver, `pipeline.py`, runs the phases in order and stores every intermediate product in the `CompileResult`; phases 1–3 form the front end, which stops compilation if it reports errors. After intermediate-code generation, each function is cut into basic blocks twice: one copy feeds an unoptimised bytecode generator, and the other flows through the optimiser, the register code generator, SSA construction, dependence analysis, the optimised bytecode generator and the LLVM IR emitter. The optimised bytecode must pass the verifier before the sandboxed VM will run it; the unoptimised bytecode and the register code are also executed, and the three outputs are compared. Four front ends render the result.

D. Lexical Analysis

Each of the 41 token patterns is an extended regular expression. The patterns are combined into a single expression $(r_1\#_1 \mid r_2\#_2 \mid \cdots \mid r_{41}\#_{41})$, in which every end marker $\#_k$ identifies a token kind, and the syntax tree of this expression yields 147 leaf positions. The functions nullable, firstpos and lastpos are computed bottom-up, and followpos is built from the two rules of the direct method [1]: for every node n and every position $i \in \mathit{lastpos}(c_1)$,

$$\mathit{followpos}(i) \supseteq \begin{cases} \mathit{firstpos}(c_2) & \text{if } n = c_1 \cdot c_2, \\ \mathit{firstpos}(c_1) & \text{if } n = c_1^{*}. \end{cases} \tag{1}$$

DFA states are sets of positions: the start state is $\mathit{firstpos}(\mathit{root})$, and the transition on input class a from state S is the union of $\mathit{followpos}(p)$ over the positions $p \in S$ that carry a. A state that contains several end markers accepts the token with the lowest index, which is how keywords take priority over identifiers. Characters that behave identically are grouped into 37 equivalence classes, which become the DFA's columns. The automaton is minimised by Moore's partition refinement [20], starting from one block per token kind plus one for non-accepting states.

Scanning simulates the minimised DFA with the maximal-munch rule: the scanner keeps reading while a transition exists, remembers the last accepting state it passed, and when it is stuck gives back whatever it read beyond that point. Python's `re` module is never imported. Each token records the list of DFA states it walked through and how many characters were read ahead and returned, which is what the studio's *scanner trace* draws (Fig. 5): the chain of states for the chosen token, the character on every edge, the accepting token of every accepting state, and the final state highlighted. Fig. 4 shows the summary view and the token stream, and Fig. 6 the minimised automaton itself.

E. Syntax Analysis

Grammar analysis. The grammar module computes FIRST and FOLLOW sets and the LL(1) table and explains every conflict. It then applies left-recursion elimination ($A \to A\alpha \mid \beta$ becomes $A \to \beta A'$, $A' \to \alpha A' \mid \varepsilon$) and left factoring, and rechecks the table. The Nova grammar is written left-recursively to obtain left-associative operators, so it is deliberately not LL(1); Section IV-B quantifies this, and Fig. 12 shows the explanations.

LR tables. From the same grammar, the LR module builds the canonical collection of LR(0) items and derives the SLR(1) table from it, using FOLLOW sets for reductions. It builds the canonical collection of LR(1) items $[A \to \alpha \cdot \beta,\ a]$ for the CLR(1) table, and builds the LALR(1) table by merging LR(1) states that share an LR(0) core:

$$I_K^{\mathrm{LALR}} = \bigcup \{\, I \in \mathcal{C}_{\mathrm{LR(1)}} \ :\ \mathrm{core}(I) = K \,\}. \tag{2}$$

Conflicts are not merely detected but explained: the module names the colliding actions, the look-ahead and the responsible productions. A deliberately ambiguous five-production expression grammar demonstrates genuine shift/reduce conflicts (Fig. 13).

The automaton, browsable. The studio exposes the LALR(1) automaton directly (Fig. 8). For any of the 116 states it lists the items, with kernel items in bold and the merged look-ahead sets in braces, and beside them the ACTION entries (shift, with the target state linked; reduce, with the production; accept) and the GOTO entries. The ACTION/GOTO table itself can be paged through eighteen states at a time (Fig. 9); only the columns that are used in the window are shown, so the sparsity of the table is visible. Because the automaton depends only on the language, it is computed once when the server starts and served to the browser as a 160 KB JSON document that is fetched a single time.

Two parsers, one language. The abstract syntax tree (AST) used by later phases is built by a hand-written recursive-descent parser with one token of look-ahead, a precedence-climbing expression chain and panic-mode recovery. On an error the parser skips to the next token that can start a statement, or to `end`, `else`, `when` or `otherwise`, so that

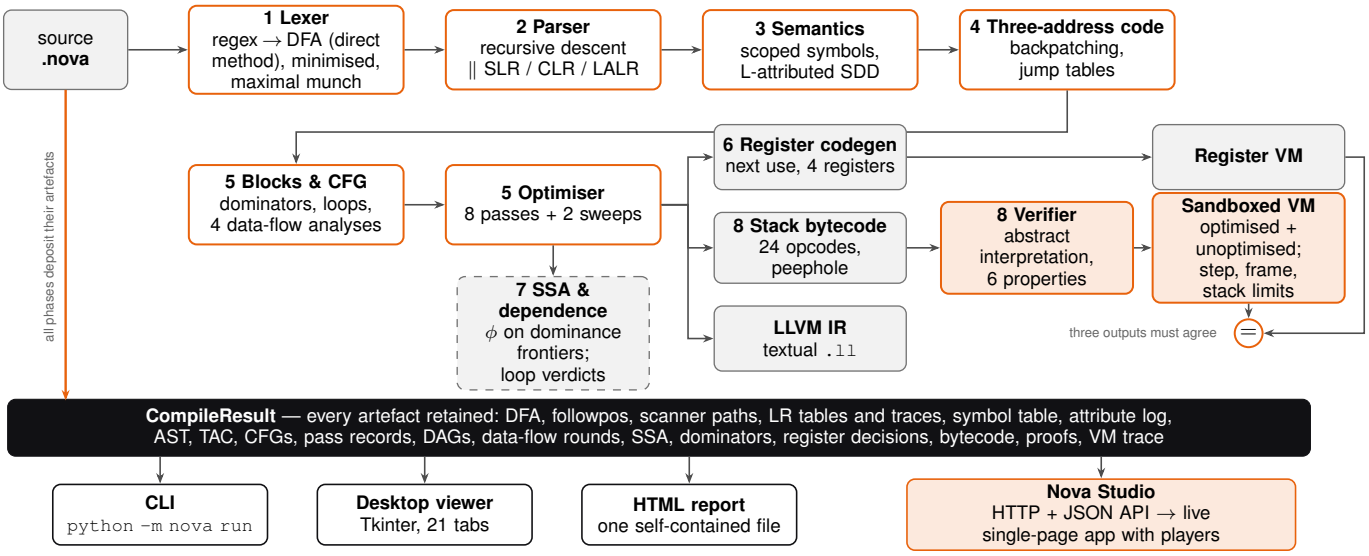


Fig. 3. Architecture of Nova. Orange-bordered boxes are compiler phases (numbered by syllabus module); grey boxes are back-ends; tinted boxes enforce safety. The optimised path is shown solid; a second, unoptimised copy of each function is compiled to bytecode and executed so that three independent executions can be compared. Every phase deposits its artefacts in a single `CompileResult`, which four front ends render.

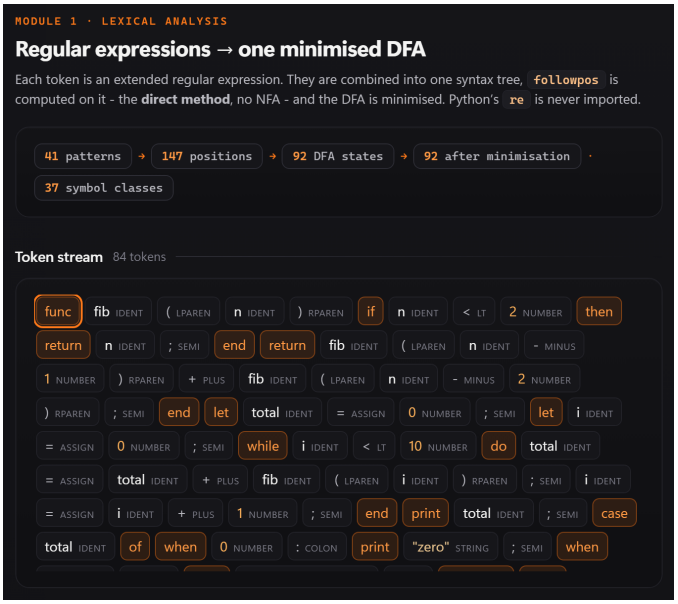
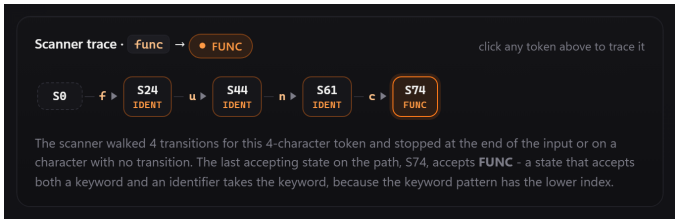
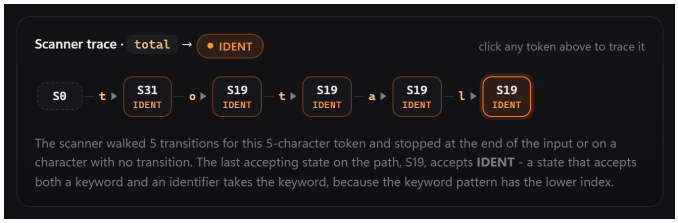


Fig. 4. Lexical view: 41 patterns yield 147 positions and a 92-state DFA (already minimal) over 37 symbol classes. Below, the token stream of the Fibonacci program, with keywords highlighted in orange and each token's kind shown in small capitals. Every token is a button that opens the scanner trace of Fig. 5.



(a) The keyword `func`: four transitions, three of them through states that accept `IDENT`, ending in state 74, which accepts `FUNC`.



(b) The identifier `total`: after the first letter the scanner stays in state 19, the generic identifier state, for the remaining four characters.

Fig. 5. The scanner trace. Clicking a token in the stream of Fig. 4 shows the exact path the minimised DFA took for it, with the input character on each edge and the token each accepting state accepts. The explanation below the chain states why scanning stopped and why the final state's token was chosen.

one missing semicolon does not cause a cascade of errors. Independently, a table-driven LALR(1) driver parses the same token stream and records the state stack, the symbol stack, the remaining input and the action at every step. Agreement between the two parsers is a self-test. Fig. 7 shows the three tables measured on the grammar and Fig. 11 the complete trace.

The parse player. The trace is also replayed by a player (Fig. 10). At any step it shows the state stack with its top highlighted, the symbol stack with the handle about to be reduced marked, the remaining input as a tape with the look-ahead first, the action taken, and the kernel items of the current state, so a learner can see *why* the table says shift or reduce. The player steps forward and back, scrubs, and autoplays at 420 ms per action; the space bar and the arrow keys drive it when it has focus.

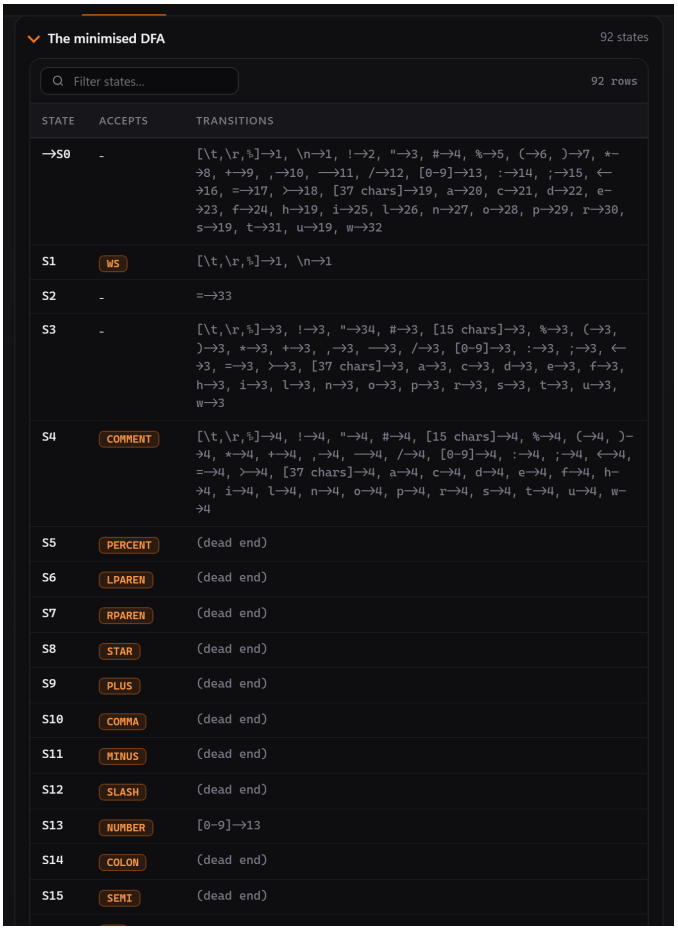


Fig. 6. The first sixteen states of the minimised DFA as the studio lists them: the start state `S0` with one transition per symbol class, the whitespace and comment loops, single-character operator states that are dead ends, and the `NUMBER` state that loops on digits.

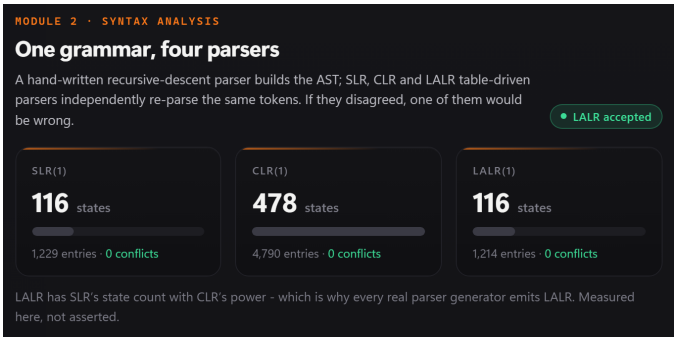


Fig. 7. Syntax view: the SLR(1), CLR(1) and LALR(1) tables for the 58-production Nova grammar, with state counts, table entries and conflicts measured on the real grammar. The LALR(1) driver accepted the program.

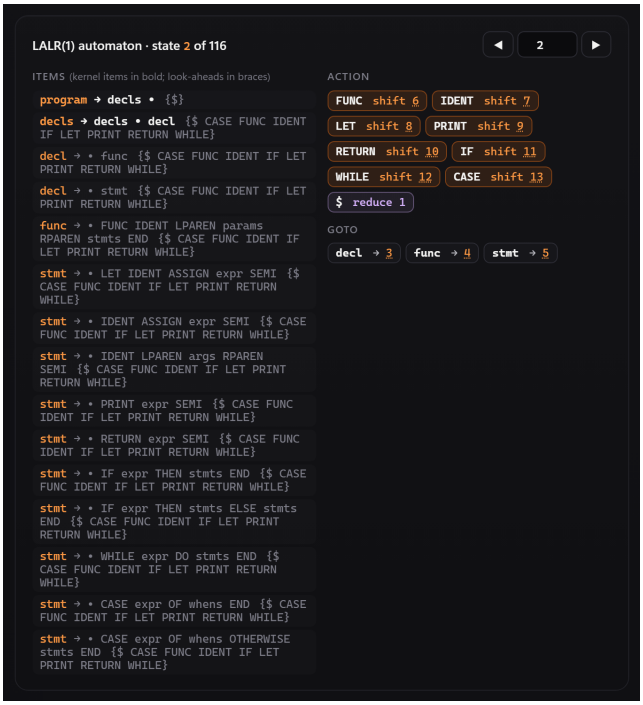


Fig. 8. The automaton browser on state 2 of the LALR(1) automaton. Left: the kernel item `program → decls •` and the closure items for every statement form, each with its merged look-ahead set. Right: the ACTION row (shift on each statement-starting terminal, reduce production 1 on end of input) and the GOTO row. Shift targets and GOTO targets are links to the next state.

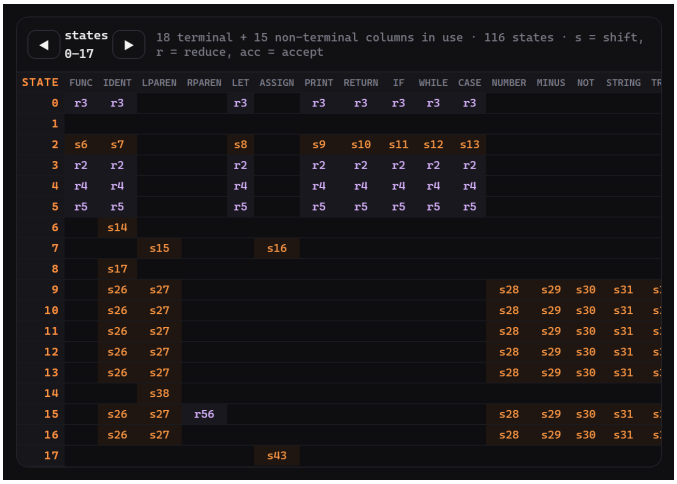


Fig. 9. The ACTION/GOTO table, states 0–17 of 116. Shift entries are orange, reduce entries violet, accept green; only the 18 terminal and 15 non-terminal columns used in this window are drawn. States 9–13 and 15–16 share one row pattern because they all expect an expression.

F. Semantic Analysis

Semantic analysis is a single L-attributed pass over the AST, preceded by a pass that collects function signatures so that calls may precede declarations. *Inherited* attributes flow downwards: the current scope, the nesting depth, the enclosing function and the type the context expects. *Synthesised* attributes flow upwards: the type of every expression. Each symbol records its name, kind, type, scope, depth, frame slot, declaring line and whether it is used. The type rules are as follows:

- arithmetic maps $int \times int \to int$, and + also maps $str \times str \to str$;
- relational operators produce *bool*, and `and`/`or`/`not` re-

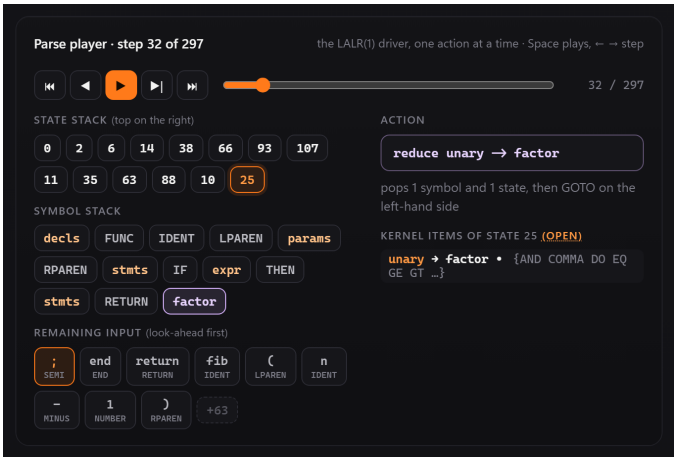


Fig. 10. The parse player at step 32 of the 297-step LALR(1) parse of the Fibonacci program. The state stack has state 25 on top; the symbol stack ends in the handle `factor` (marked), which the action `reduce unary → factor` pops before a GOTO on `unary`; the tape shows the look-ahead `;` and the next tokens; the kernel item of state 25 explains the reduction.

quire *bool*;

- every condition must be *bool*, and every argument must be *int*.

A poisoned $\langle error \rangle$ type suppresses cascades. Unused variables produce warnings rather than errors. Every attribute flow is logged per construct (Fig. 14), and the annotated tree is drawn (Fig. 15): each node carries the type the checker synthesised for it, and the drawing can be switched between the functions and the statements of the main program. The layout is a simplified Reingold–Tilford tree layout [57]: subtrees are measured bottom-up, placed left to right without overlap, and every parent is centred over its children.

G. Intermediate-Code Generation

Three-address code is generated as quadruples (*op*, arg_1, arg_2, *result*) by a syntax-directed translation scheme. Boolean expressions are compiled as *jumping code* with true and false lists, so `a and b` never evaluates `b` when `a` is false. Jump targets that are unknown when a jump is emitted are left blank and filled in by `makelist`, `merge` and `backpatch` [1], and every backpatch is logged (Fig. 17). A `case` statement is lowered to a genuine jump table (`jtab`) with a default target rather than to a chain of comparisons (Fig. 16).

H. Control Flow and Optimisation

Leaders (the first instruction, every jump target and every instruction after a jump) delimit basic blocks, and the blocks form the control-flow graph (CFG). Inside the CFG every jump names a *block* rather than an instruction index, so a pass may insert or delete instructions without invalidating jumps. Dominators are computed by iterative set intersection, immediate dominators are derived from them, and every back edge $n \to h$ with h dom n defines a natural loop. The optimiser runs eight passes in a fixed order and then repeats the first and the sixth as clean-up sweeps:

TABLE III
THE FOUR DATA-FLOW ANALYSES

Analysis	Direction	$\wedge$	Transfer $f_B(x)$
Reaching definitions	forward	$\cup$	$gen_B \cup (x - kill_B)$
Live variables	backward	$\cup$	$use_B \cup (x - def_B)$
Available expressions	forward	$\cap$	$egen_B \cup (x - ekill_B)$
Very busy expressions	backward	$\cap$	$euse_B \cup (x - ekill_B)$

1) local value numbering and DAG construction (common subexpressions, constant folding, algebraic identities);
2) global constant propagation and
3) global copy propagation, both driven by reaching definitions;
4) loop-invariant code motion (LICM) into a synthesised preheader;
5) strength reduction of induction variables;
6) dead-code elimination driven by liveness;
7) unreachable-code elimination; and
8) jump optimisation.

Each pass records the instruction count before and after it and the code it produced, which the studio plots as a waterfall (Fig. 18). Flow graphs are drawn before and after optimisation (Fig. 19) in a layered layout in the manner of Sugiyama [58]: blocks are assigned to layers breadth-first from the entry, forward edges are cubic Bézier curves, and back edges are routed outside the blocks as dashed orange curves. The DAG of any block is drawn as well (Fig. 20): leaves at the bottom, operators above their operands, each node annotated with the variables that hold its value, and nodes with more than one parent—the common subexpressions—outlined in orange.

I. Data-Flow Analysis

Four classical analyses are solved over the CFG in the monotone framework of Kildall [30]. With $\wedge$ the meet operator and f_B the transfer function of block B, a forward analysis computes

$$IN[B] = \bigwedge_{P \in \mathrm{pred}(B)} OUT[P], \quad OUT[B] = f_B(IN[B]), \tag{3}$$

and a backward analysis computes

$$OUT[B] = \bigwedge_{S \in \mathrm{succ}(B)} IN[S], \quad IN[B] = f_B(OUT[B]), \tag{4}$$

with the instances listed in Table III. Reaching definitions drive constant and copy propagation, live variables drive dead-code elimination and register allocation, available expressions support global common-subexpression elimination, and very busy expressions justify code hoisting. Each analysis is solved by round-robin iteration over the blocks until no IN or OUT set changes, and a snapshot of every round is kept, so the convergence itself can be inspected (Fig. 22). The studio also writes the final IN and OUT sets of the chosen analysis into the blocks of the flow graph (Fig. 21), which places each fact next to the code it is a fact about.

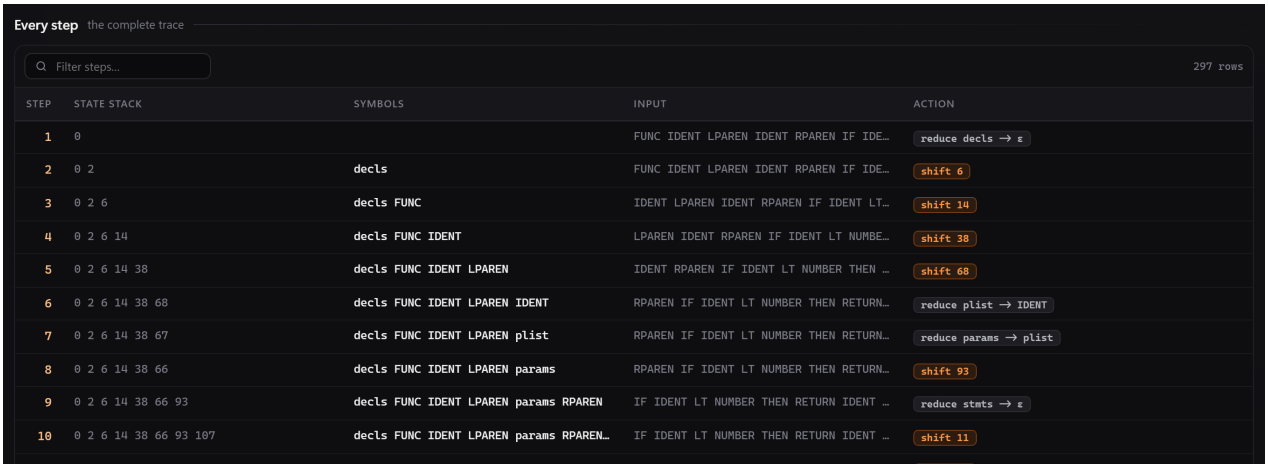


Fig. 11. The first ten of 297 steps of the LALR(1) parse of the Fibonacci program, as the studio tabulates them: state stack, symbol stack, remaining input and the shift or reduce action taken. The filter box searches all steps.

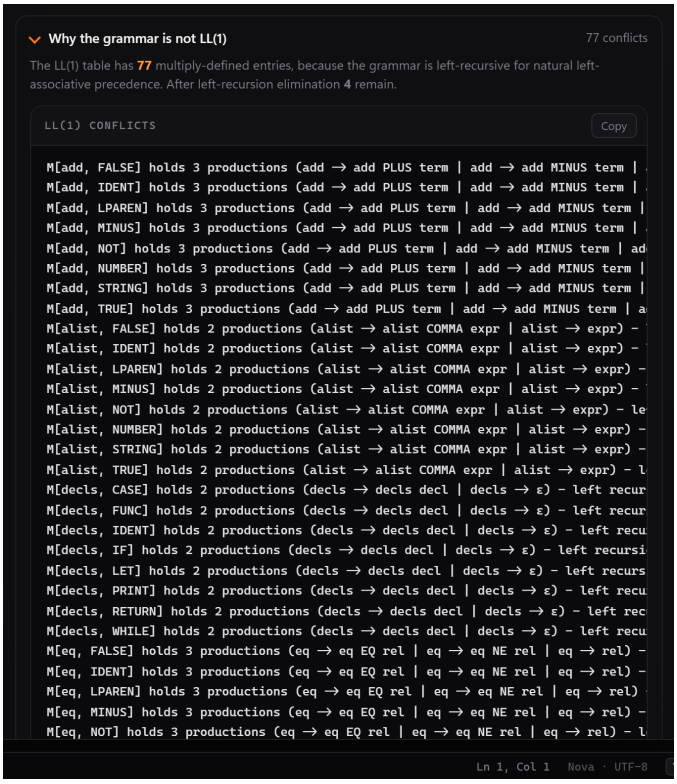


Fig. 12. Why the grammar is not LL(1): the first of 77 multiply defined table entries, each naming the productions that collide and the reason—left recursion—and the count of 4 that remain after left-recursion elimination.

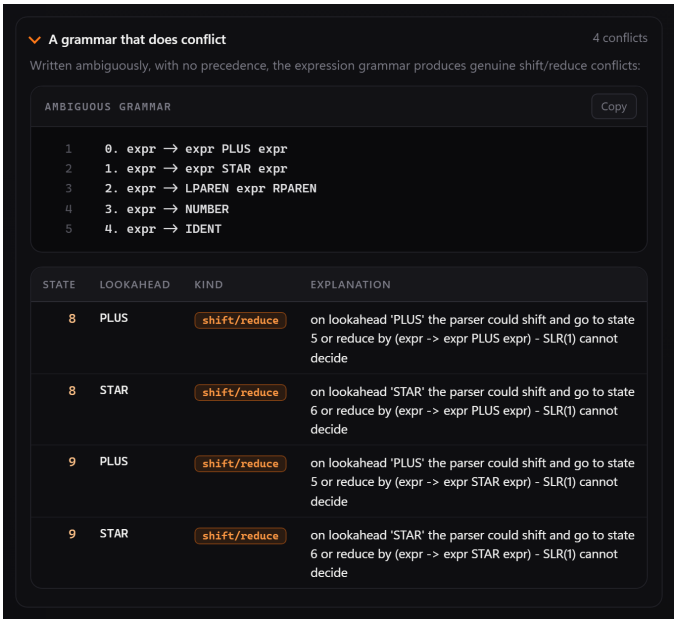


Fig. 13. A grammar that does conflict. The five-production expression grammar without precedence produces four shift/reduce conflicts under SLR(1) construction, two in each of states 8 and 9 on + and ⋆; the explanation names the items that collide.

J. Code Generation and Register Allocation

The register code generator implements the textbook simple code generator over four general-purpose registers. A backward scan of each block computes, for every instruction, the next use and liveness of each name, starting from the block's live-out set. The register-selection routine `getreg` chooses, in order: an empty register; a register whose value is dead; or a register to spill, chosen by Belady's rule [43],

$$r^{*} = \arg\max_{r} \; nextuse\big(v(r)\big), \tag{5}$$

where $v(r)$ is the value held in register r. A register descriptor (what each register holds) and an address descriptor (where each value lives) are recorded after every instruction (Fig. 25), and values that are live at a block's exit are stored to their slots in the activation record, so that blocks can be generated independently. The activation record holds the return address at offset -2, the control link at -1, then parameters, locals and temporaries (Fig. 23). A register-machine simulator executes the generated code, which turns the code listing into a working

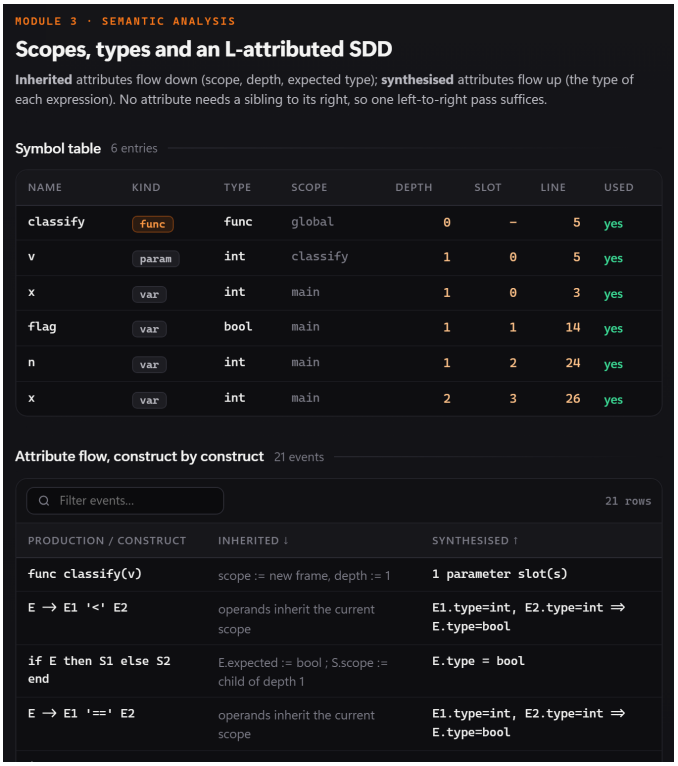


Fig. 14. Semantic view of `07_types_and_scopes.nova`: the scoped symbol table, in which the inner `x` at depth 2 shadows the outer one, and the attribute flow construct by construct, with inherited attributes passed down and synthesised attributes passed up.

back-end. The studio turns the register descriptors into a timeline (Fig. 24): one row per register, one column per instruction, a coloured run for every interval during which a value occupies a register, so that the allocator's reuse of registers and its spills can be read at a glance.

K. SSA Construction and Parallelism

SSA form is built by the method of Cytron *et al.* [37]. The dominance frontier of block n is

$$\mathrm{DF}(n) = \{\, y \mid \exists p \in \mathrm{pred}(y) : n \text{ dom } p \ \wedge\ \neg(n \text{ sdom } y) \,\}, \tag{6}$$

computed by walking up the dominator tree from the predecessors of each join block. The entry block is treated as having an extra, implicit predecessor—the function's start—so a loop that begins at the very first instruction is still recognised as a join (Section IV-K). ϕ-functions are placed on the iterated dominance frontier of each variable's definition sites, and variables are renamed in a pre-order walk of the dominator tree using one version stack per variable (Fig. 26). The dominator tree itself is drawn, with each block's dominance frontier beside it (Fig. 27).

For every natural loop, the dependence analyser computes four sets of variables:

- W, those written in the loop;
- U, the upward-exposed reads (read before being written in an iteration);
- I, the basic induction variables ($x = x + c$);
- $C = (W \cap U) - I$, the carried dependences, within which $R \subseteq C$ are the reductions matching $x = x \oplus e$ with $\oplus \in \{+, \times\}$.

A loop with $C - R = \emptyset$ and $R = \emptyset$ is *vectorisable*; one with $C - R = \emptyset$ and $R \neq \emptyset$ is *parallelisable as a reduction*, as OpenMP's `reduction` clause allows [49]; any other loop is *sequential*. Variables in $W - U - I$ are reported as privatisable, and for a loop nested inside another the analyser adds the two classical locality transformations a real compiler would consider next, interchange and tiling (Figs. 28 and 29).

L. Bytecode, Verification and Sandboxed Execution

Bytecode. The second back-end emits a JVM-like stack bytecode of 24 opcodes:

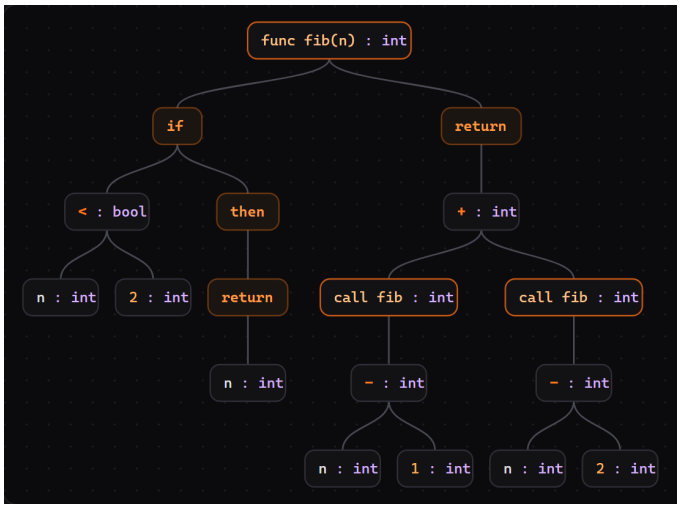
- constants, `LOAD` and `STORE`;
- arithmetic and `NEG`;
- `DUP` and `POP`;
- six conditional branches, `GOTO` and `SWITCH`;
- `PRINT`, `CALL`, `RETURN` and `HALT`.

A peephole pass [36] rewrites `STORE x; LOAD x` as `DUP; STORE x`, removes jumps to the next instruction and cancels double negation (Figs. 30 and 31).

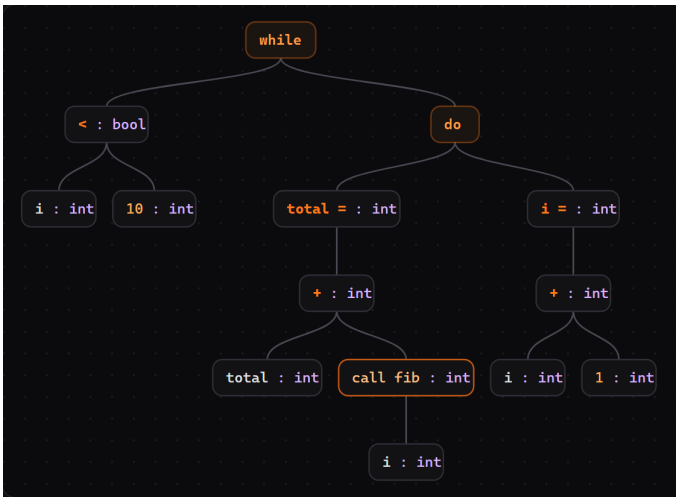
Verification. Before anything runs, the verifier performs abstract interpretation [50] over each function's bytecode (Algorithm 1). The abstract state at instruction i is $\sigma_i = (\tau, D)$, where τ is a stack of abstract types drawn from $\{int, str, any\}$ and D is the set of definitely initialised local slots. At a merge point the stack depths must be equal; the types are joined pointwise (equal types are kept, unequal ones become any), and the sets D are intersected. Because depths are fixed at merges, types only move towards any and D only shrinks, the iteration terminates. The verifier proves six properties: (P1) a consistent operand-stack depth at every merge, (P2) type safety of every instruction, (P3) every branch target is inside the code, (P4) definite assignment, (P5) no `LOAD`/`STORE` outside the activation record, and (P6) no unreachable instructions (reported as a warning). It also records the proved stack depth at every instruction, which the studio serves with the verification report.

Sandboxed execution. The VM refuses unverified code. A call pushes a frame holding the callee's locals, its operand stack, the return address and a control link to the caller's frame. Execution runs under hard limits of 2,000,000 instructions, 256 frames and 1,024 operand-stack entries, so a non-terminating or runaway program is reported as a policy violation instead of hanging the tool. The VM records the first 400 steps as a trace—each step with its function, program counter, instruction, operand stack, call depth and the locals of the current frame—and keeps a snapshot of the activation-record stack at its deepest (Figs. 41 and 37). A hostile-bytecode suite of eight hand-written programs, discussed in Section IV-H, exercises the verifier and the sandbox together.

The VM stepper. The studio replays the trace as a stepper (Fig. 38): the bytecode of the current function with the



(a) The function `fib`: the condition synthesises `bool`, every arithmetic node and call synthesises `int`.



(b) The `while` loop of the main program: its condition and body, with the recursive call inside the accumulation.

Fig. 15. The annotated syntax tree of the Fibonacci program as the studio draws it. Keywords are orange-bordered, operators orange, literals amber, and the synthesised type attribute follows each node's label in violet.



Fig. 16. Three-address code for the main program of the Fibonacci benchmark: the loop as backpatched conditional jumps, and the `case` statement as a single jump table `[0→L3, 88→L4] default L5`.



Fig. 17. Every backpatch of the Fibonacci compilation: each line fills a list of pending jumps with the address that has just become known; the jump table of the `case` statement is filled with its two labels and its default.

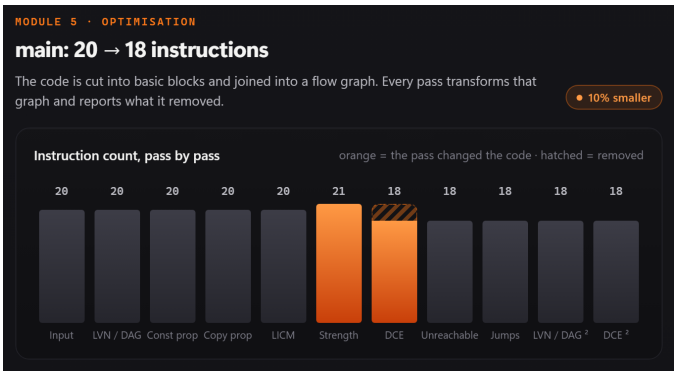


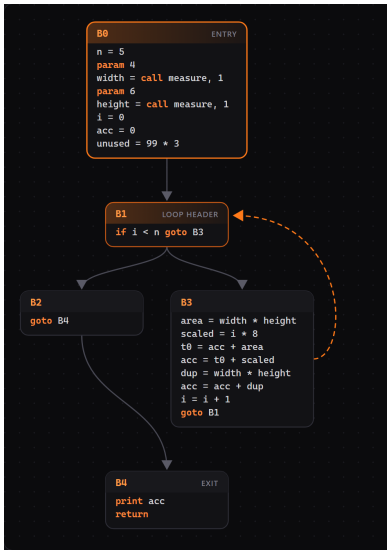
Fig. 18. Optimiser view for `05_optimiser.nova`: the instruction count of `main` after each pass. Orange bars changed the code; the hatched segment is removed code. Strength reduction temporarily adds an instruction; dead-code elimination then removes three.

program counter highlighted, the operand stack top first, the locals of the current frame by name, the call depth against the deepest reached, and the output produced so far. It steps, scrubs and autoplays at 260 ms per instruction, and answers the same keys as the parse player.

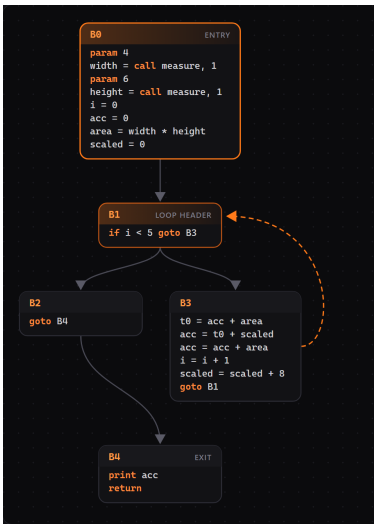
LLVM IR. A third back-end prints textual LLVM IR in the style of `clang -O0`: every variable becomes an `alloca`, and reads and writes become `load` and `store` (Fig. 32). LLVM's `mem2reg` pass would produce the same SSA form that Nova builds itself. The output needs no LLVM installation, but anyone who has `clang` can compile it to native code.

M. Nova Studio

Architecture. Nova Studio is a local web application served by a standard-library `ThreadingHTTPServer` [63]. A JSON view module mirrors the HTML report: it renders a



(a) Before optimisation



(b) After optimisation

Fig. 19. Control-flow graphs of `main` in `05_optimiser.nova`, rendered by the studio. Blocks are laid out in breadth-first layers from the entry. The entry, loop-header and exit blocks are labelled, and the dashed orange edge is the back edge that defines the loop. After optimisation, constant propagation has replaced `n` by `5` and its definition has gone, as has the never-read `unused`; the loop-invariant product `width*height` is computed once before the loop and reused instead of being recomputed as `dup`; and `scaled` is updated by addition (`scaled = scaled + 8`) instead of multiplication.

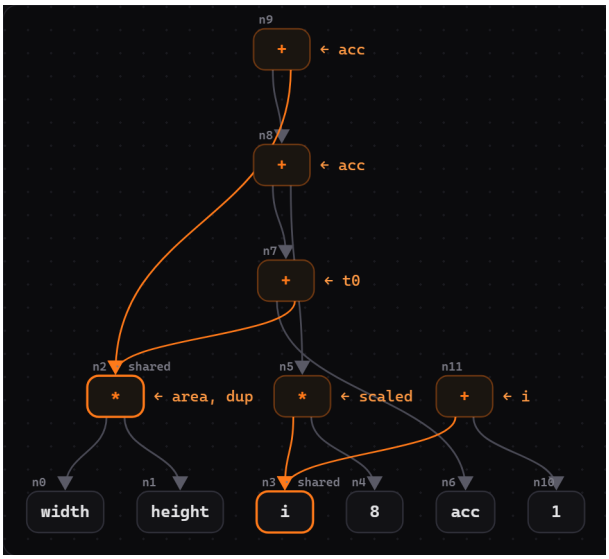


Fig. 20. The DAG of the loop body (`B3`) of `05_optimiser.nova` before optimisation. Node `n2` (`width*height`) is shared by `area` and `dup`, which is how value numbering finds the common subexpression; the leaf `i` is shared by `scaled = i*8` and `i = i+1`. Orange edges lead to shared nodes.

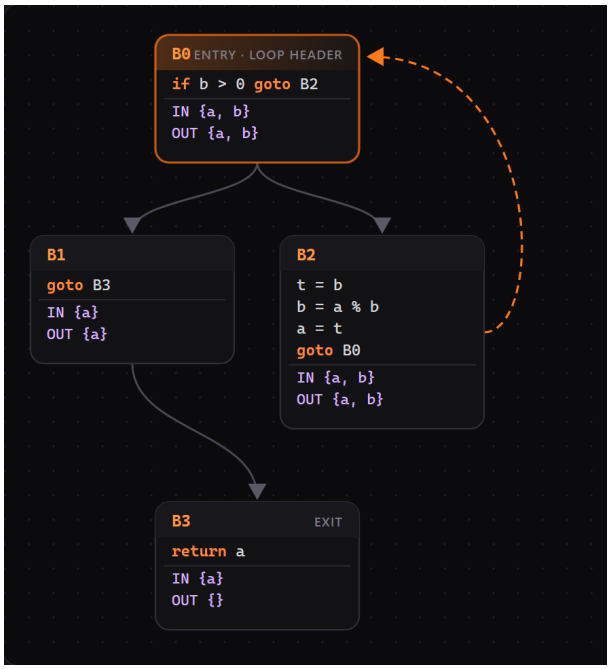


Fig. 21. Live-variable facts written onto the flow graph of `gcd`: both `a` and `b` are live into the loop header and through the loop body, only `a` is live on the exit path, and nothing is live after `return`.

`CompileResult` as plain dictionaries and lists (Table IV), and it caps long lists at 2,500 rows while recording their true lengths. The API is summarised in Table V. The client is a single page written in plain JavaScript (1,846 lines) and CSS (837 lines), with no frameworks and no build step; it is served with a content hash in its URLs so that a browser never runs a stale copy.

Live compilation. The editor sends a compile request 360 ms after the user stops typing. The client keeps only the latest request: a new one aborts the request in flight, and a monotonically increasing sequence number discards stale responses. The server serialises compilations with a lock and builds the LR tables at start-up, before the first request. While the user types, each of the three executions receives a budget of 250,000 instructions, so an unfinished loop cannot freeze the page.

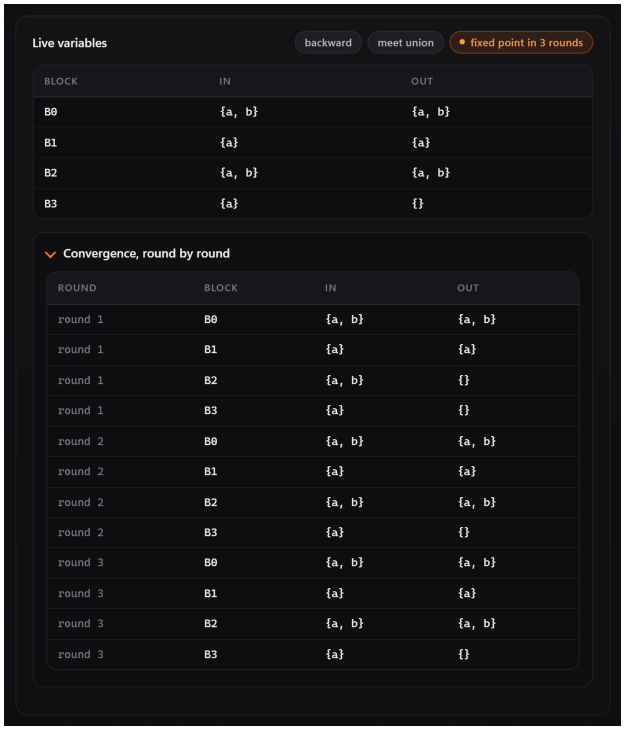


Fig. 22. Live-variable analysis of `gcd`: the final IN/OUT sets and the convergence table. `B2`'s OUT set is empty after round 1 and becomes $\{a, b\}$ in round 2 once the loop's back edge has propagated it; round 3 confirms the fixed point.

Algorithm 1 Bytecode verification by abstract interpretation

Require: function code $c_0..c_{n-1}$ with p parameters and s slots
Ensure: a verdict for each of the properties P1–P6

$\sigma_0 \leftarrow (\langle\,\rangle, \{0, \ldots, p-1\})$; $W \leftarrow \{0\}$
while $W \neq \emptyset$ **do**
  remove some i from W; $(\tau, D) \leftarrow \sigma_i$
  $(\tau', D') \leftarrow \mathit{transfer}(c_i, \tau, D)$ {P2, P4, P5}
  for all $j \in \mathit{succ}(i)$ **do**
    if $j \notin [0, n)$ **then**
      report a branch outside the code {P3}
    else if σ_j is undefined **then**
      $\sigma_j \leftarrow (\tau', D')$; $W \leftarrow W \cup \{j\}$
    else if $|\tau_j| \neq |\tau'|$ **then**
      report inconsistent stack depth {P1}
    else
      $\sigma'' \leftarrow (\tau_j \sqcup \tau',\ D_j \cap D')$
      if $\sigma'' \neq \sigma_j$ **then**
        $\sigma_j \leftarrow \sigma''$; $W \leftarrow W \cup \{j\}$
      end if
    end if
  end for
end while
report every instruction never reached {P6}

Pressing *Run* (`Ctrl+Enter`) restores the normal 2,000,000, and the studio says which budget stopped a program.

Instruments. Beyond the tables and code listings, the inspector contains the interactive instruments listed in Table VI, each of which has been shown in the preceding subsections. All of them are drawn in the browser from the JSON document: the trees and the dominator tree by the tidy layout, the flow graphs and DAGs by layered layouts, the timeline as a table of coloured runs, and the two players as state machines over the trace rows. The three drawings share one layout engine, and the two players share one controller, so adding an instrument is a matter of a view function.

Interface. The interface comprises the following components:

- a syntax-highlighting editor whose gutter marks diagnostics;
- the nine-stage pipeline rail;
- an inspector with fourteen views and a per-function selector;
- filterable tables;
- a problems panel that jumps to the offending line (Fig. 44);

TABLE IV
THE COMPILE JSON DOCUMENT

Key	Contents
`diagnostics`	phase, line, column, message, hint, severity
`lexical`	DFA statistics, tokens, followpos rows, DFA rows, and per token the DFA path and read-ahead
`syntax`	the three tables' sizes, the LALR(1) verdict and trace, the recursive-descent trace
`semantic`	symbol rows, attribute-flow rows, the annotated tree as text and as nested nodes
`functions[]`	per function: TAC, CFG before and after, pass records, DAG nodes, four data-flow solutions with rounds, frame layout, next-use rows, register decisions, target code, dominators, SSA blocks, loop reports
`bytecode`	listing, per-function code and slots, string pool, peephole notes, instruction counts
`verifier[]`	per function: six checks, errors, proved stack depth at every instruction
`run`	output, error, steps, frames, trace rows with locals, the other two back-ends, agreement
`summary`, `timings`, `metrics`	the headline numbers and the measurement table

TABLE V
NOVA STUDIO HTTP API

Endpoint	Purpose
`GET /api/health`	liveness, version and execution budgets
`GET /api/examples`	the eleven example programs
`GET /api/grammar`	grammar, FIRST/FOLLOW, LL(1) conflicts, ambiguity demonstration, DFA accepting states, the LALR(1) automaton and its ACTION/GOTO table (computed once)
`POST /api/compile`	`{source, optimise, mode}` → every phase as JSON
`POST /api/report`	the self-contained HTML report as a download
`GET /api/attacks`	hostile bytecode against verifier and sandbox

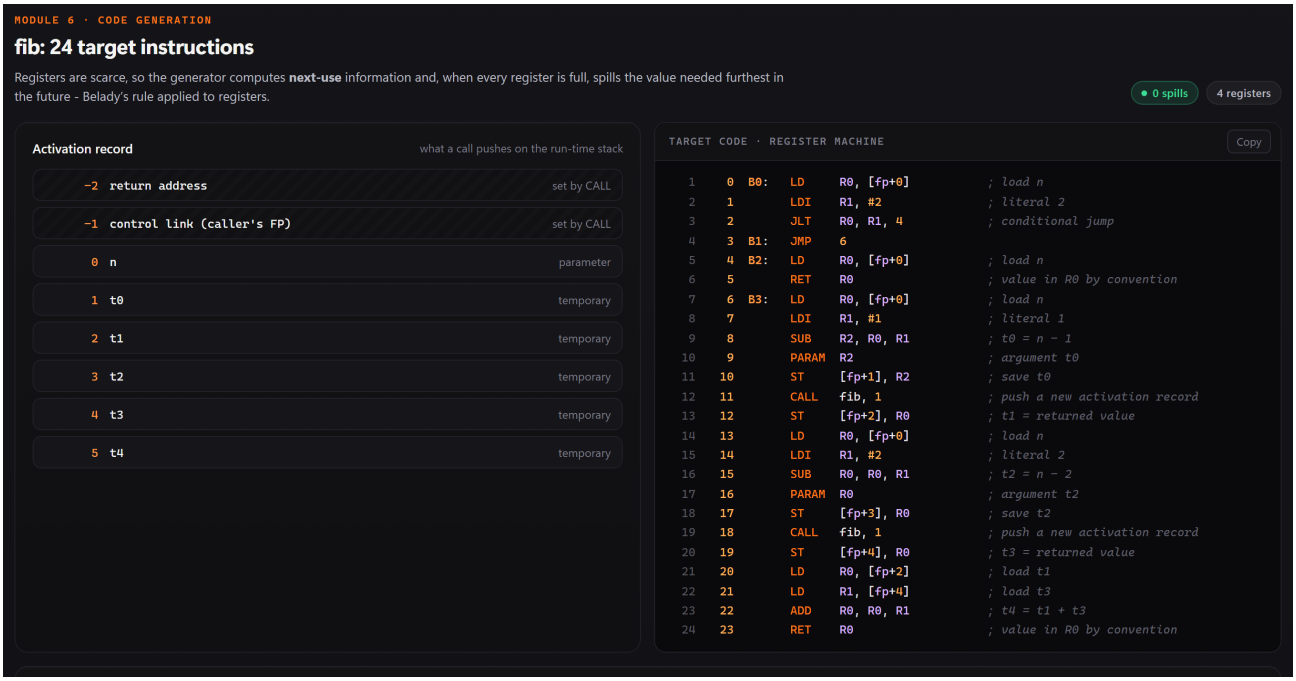


Fig. 23. Code-generation view for the recursive function `fib`: the activation-record layout (left) and the 24-instruction register-machine code (right), with the allocator's comment on each instruction. No spills are needed with four registers.

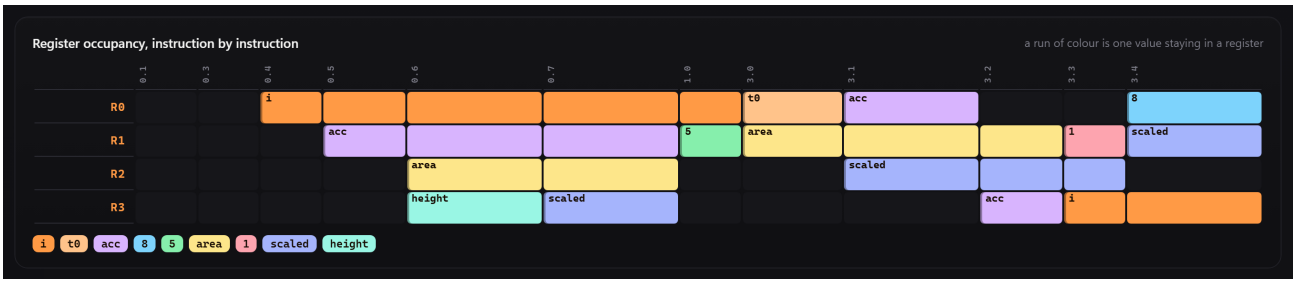


Fig. 24. Register occupancy for `main` of `05_optimiser.nova`, instruction by instruction. Each coloured run is one value held in one register; `i` stays in `R0` through the whole entry block, `area` moves from `R2` to `R1` between blocks because every block is generated independently, and the constants `5`, `1` and `8` occupy registers only for the instruction that needs them.

- a syllabus map that connects each module of the course to the views that demonstrate it, with the numbers of the current program (Fig. 33);
- deep links of the form `#tab=optimiser&fn=main`, and *share* links that carry the whole program in the URL, so that a particular program and view can be sent to another person;
- downloads of the HTML report and of the compile JSON;
- keyboard shortcuts (Fig. 34), and a layout that adapts down to a 390-pixel-wide phone screen (Fig. 35).

Security model. A development server on `localhost` can be reached by any web page that the user visits. Nova Studio therefore applies six controls:

- it binds to the loopback interface;
- it accepts only `application/json` POST bodies, which forces a cross-origin preflight that the server never grants [64];
- it rejects requests whose `Host` header does not name the loopback interface, which defeats DNS rebinding [65];
- it caps request bodies at 512 KB;
- it sends a restrictive Content Security Policy [66] and `nosniff` headers;
- it versions its assets by content hash, so browsers never execute stale code.

N. Evaluation Methodology

Benchmarks. The suite in Table VII contains eleven programs, each chosen to exercise particular phases; `08_broken.nova` contains one deliberate mistake per front-end phase, and `11_deep_recursion.nova` recurses 200 deep, inside the sandbox's limit of 256 frames. A synthetic series of programs made of 1 to 64 copies of a seven-line loop measures how compile time scales with program size.

Metrics and measurement protocol. Structural metrics are read from the `CompileResult`. Compile times are medi-



Fig. 25. The allocator's decisions for `main` of `05_optimiser.nova`: after every instruction, the register descriptor (what each register holds) and the address descriptor (where every value currently lives—a register, memory or both). At `B3.0`, for example, `t0` is created in `R0` while `area` lives in both `R1` and memory.

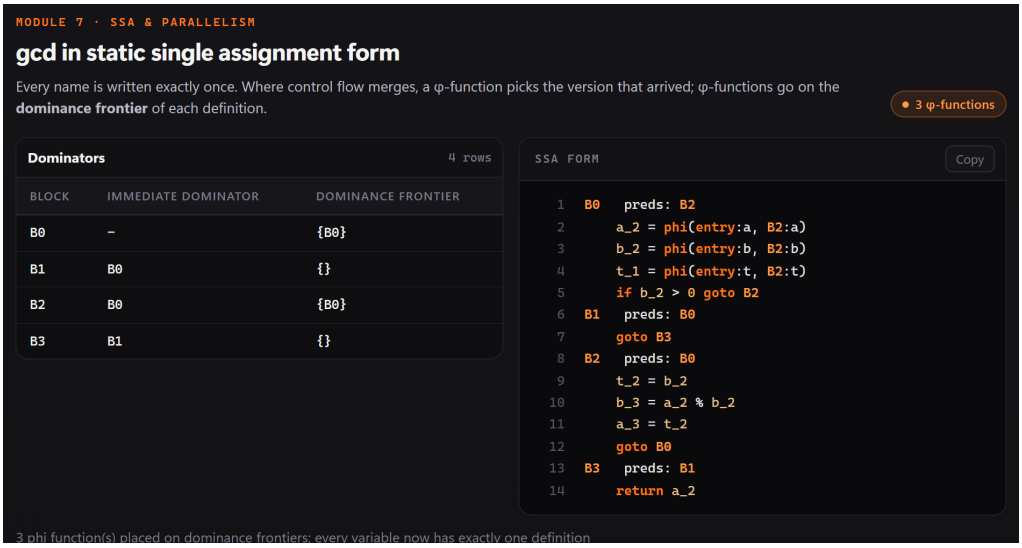


Fig. 26. SSA view of `gcd`. Its loop header is also the entry block, so `B0` is in its own dominance frontier and in that of `B2`. The ϕ-functions merge each variable's value on entry with the value from the back edge, and the condition and `return` read the ϕ results.

ans of 60 warm runs measured with a high-resolution wall clock and cross-checked against process CPU time; LR table construction times are medians of three runs with a fresh builder; scaling times are medians of 25 runs (12 for the two largest programs); HTTP round trips are medians of 30 requests over one keep-alive connection to a separate server process, which is how a browser uses the studio. The measurement script records the machine's power state, because a laptop on battery runs its processor more slowly. The script `report/tools/measure.py` produces every number in Section IV; `report/tools/capture.mjs` produces every studio screenshot by driving Nova Studio through the Chrome DevTools protocol at twice the display density, and `report/tools/desktop_capture.ps1` photographs the desktop viewer's own window.

Correctness. A self-test suite of 34 tests (Table VIII) combines four kinds of evidence:

- textbook checks, for example that the DFA for $(a|b)^*abb$ has four states;
- differential tests across the three back-ends and with the optimiser on and off, for all ten valid programs;
- the hostile-bytecode suite;
- HTTP API tests and regression tests for every defect reported in Section IV-K.

Every view of the studio is also exercised for every example program by a headless browser that records any page error; the capture of the figures in this report doubled as that test, and no error was recorded.

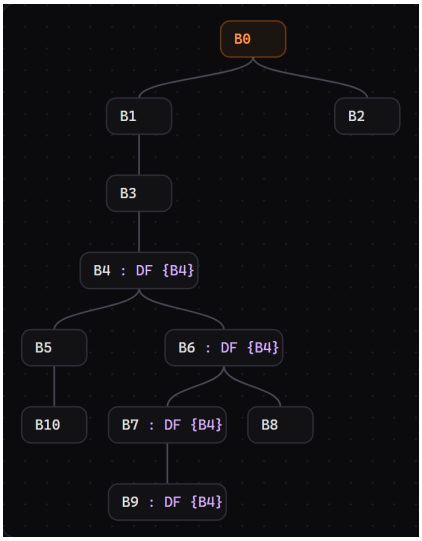


Fig. 27. The dominator tree of `isprime` in `04_primes.nova`. The loop header `B4` dominates the whole loop and is in its own dominance frontier; the blocks inside the loop that can reach the header again (`B6`, `B7`, `B9`) have `B4` in their frontier too, which is why the ϕ-functions for `d` and the temporaries are placed in `B4`.

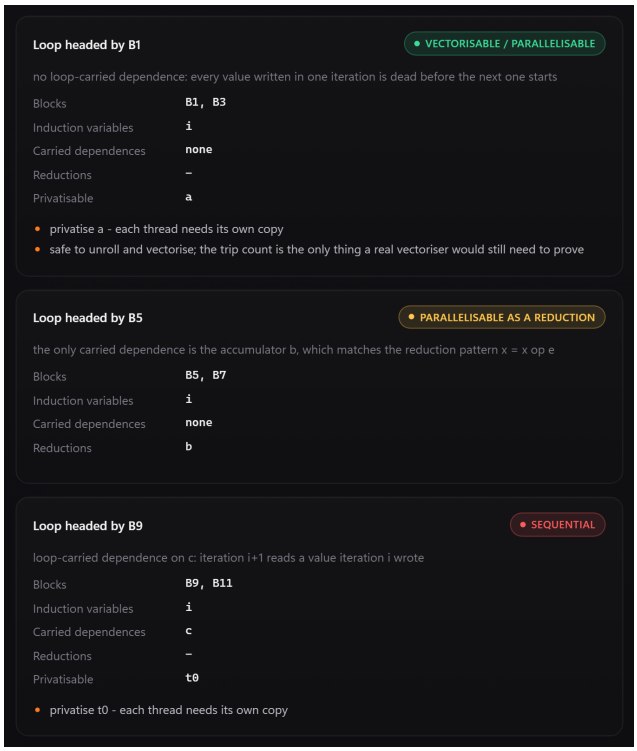


Fig. 28. Loop dependence analysis of `06_parallel.nova`. Three loops that look alike are classified as vectorisable (no carried dependence; `a` is privatisable), parallelisable as a reduction (accumulator `b`) and sequential (iteration $i+1$ reads the value of `c` written by iteration i).

IV. Results and Discussion

A. Experimental Setup

All measurements were taken on a laptop with a 13th-generation Intel Core i5-13500HX processor (14 cores, 20

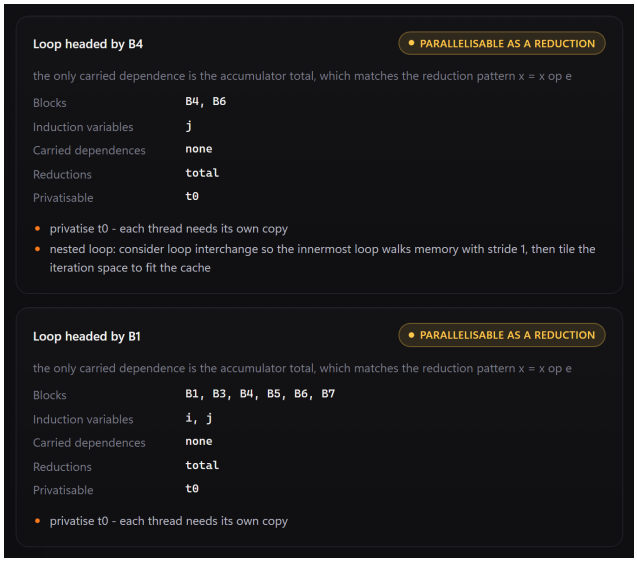


Fig. 29. The nested loop pair of `09_nested_loops.nova`. Both loops are reductions on `total`; the inner loop additionally receives the locality advice—interchange so that the innermost loop walks memory with stride one, then tile—that a nested loop earns.



Fig. 30. The stack bytecode of the Fibonacci program: the string pool, then `fib` with its parameter and slot counts. The verifier's maximum stack depth of 2 is visible in the code—no instruction ever needs more than two operands.



Fig. 31. Peephole optimisations applied to the final bytecode of the Fibonacci program: three `STORE x; LOAD x` pairs rewritten as `DUP; STORE x`.



Fig. 32. The textual LLVM IR for the Fibonacci program: every Nova variable is an `alloca` slot, every read a `load` and every write a `store`, exactly as `clang -O0` would emit it.

threads), 15.7 GB of memory and Windows 11 (build 26200), using CPython 3.14.3. The laptop was running on *battery* under its vendor power plan, which throttles the processor; an earlier session on mains power ran the same compilations roughly 3.5× faster. All timings in this section are therefore conservative, and the ratios between them, not the absolute values, are the durable findings. Screenshots were rendered by Chrome 155 driven from Node.js 24. All 34 self-tests pass.

B. Lexical and Syntactic Analysis

Table IX summarises the front end. The direct method produces a 92-state DFA that Moore's refinement cannot reduce further: building the automaton from position sets yields no equivalent states for this specification. The scanner trace confirms the intended priority rule on real tokens: in Fig. 5a the path of `func` passes through three states that accept `IDENT` before ending in state 74, which accepts both `IDENT` and `FUNC` and reports `FUNC` because the keyword pattern has the lower index, whereas `total` (Fig. 5b) enters the generic identifier state 19 after its first character and stays there.

The Nova grammar has 77 multiply defined LL(1) entries, most of which come from left recursion. After left-recursion elimination the grammar grows to 69 productions

TABLE VI
INTERACTIVE INSTRUMENTS IN THE INSPECTOR

Instrument	Mod.	What the learner can do
Scanner trace	1	click a token; see its DFA path, edges and accepting states
Automaton browser	2	step through all 116 states; follow shift and GOTO links; page the table
Parse player	2	replay 297 actions with both stacks, the handle and the look-ahead tape
Syntax tree	3	switch between functions and statements; read synthesised types
Pass waterfall	5	see what each of the ten passes removed or added
Flow graph	5	toggle before/after; read loops from back edges
DAG	5	choose a block; see shared nodes and attached variables
Data-flow overlay	5	choose an analysis; read IN/OUT on the graph
Register timeline	6	see every value's stay in every register
Dominator tree	7	read immediate dominators and frontiers
VM stepper	8/9	step the machine; watch stack, locals, depth, output
Syllabus map	all	see each module's requirement, implementation and numbers

TABLE VII
BENCHMARK PROGRAMS

Program	LOC[a]	Tokens	Fn.	Exercises
`hello`	4	18	1	smallest complete program
`fibonacci`	16	84	2	recursion, loop, jump table
`gcd`	13	68	3	nested calls; loop at function entry
`primes`	20	103	2	nested loops, short-circuit `and`
`optimiser`	18	97	2	CSE, LICM, strength reduction, DCE
`parallel`	23	102	1	three loops with different dependences
`types`	22	104	2	shadowing, scopes, booleans
`broken`	7	33	0	one error per front-end phase
`nested`	13	60	1	a nested loop pair; locality advice
`strings`	19	88	2	strings, booleans, `if`/`else` in a function, `otherwise`
`recursion`	10	62	3	200 nested activation records

[a]Non-blank, non-comment lines; "Fn." counts compiled functions including `main`. Program `x` is the file `examples/NN_x.nova`; `types` is `07_types_and_scopes.nova`, `nested` is `09_nested_loops.nova` and `recursion` is `11_deep_recursion.nova`.

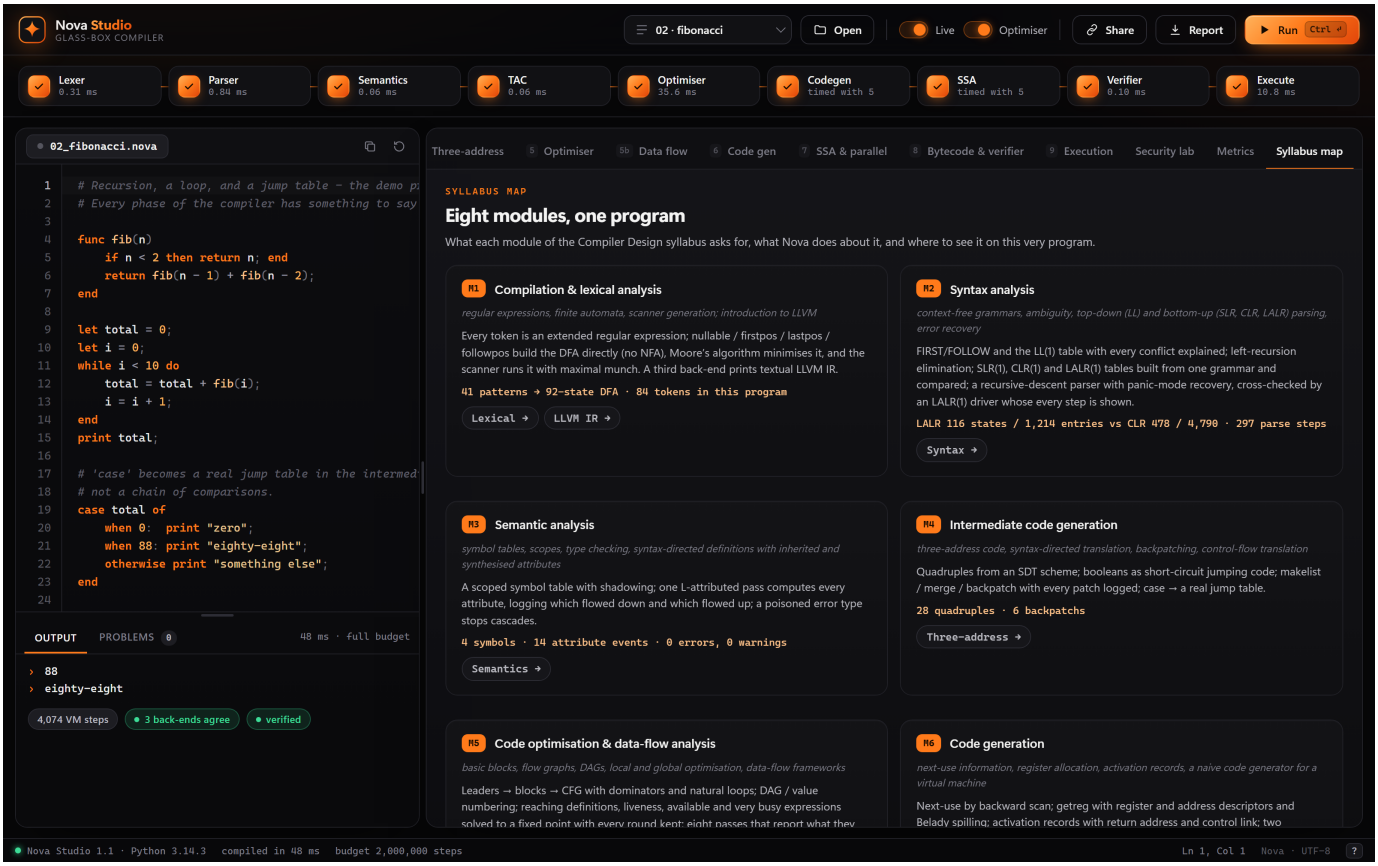


Fig. 33. The syllabus map. For each of the eight modules it states what the syllabus asks for, what Nova does about it, the measurements of the current program (here `02_fibonacci.nova`) and links to the views that show it.

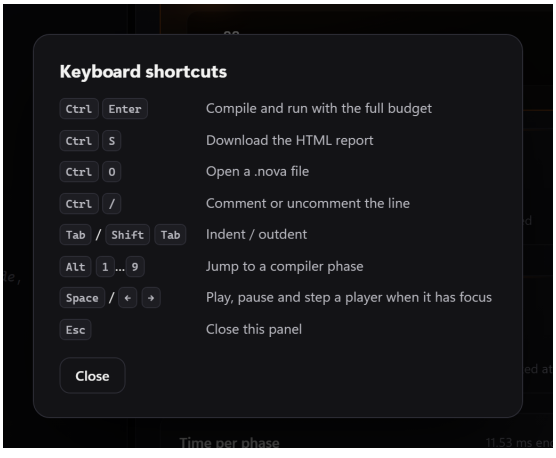


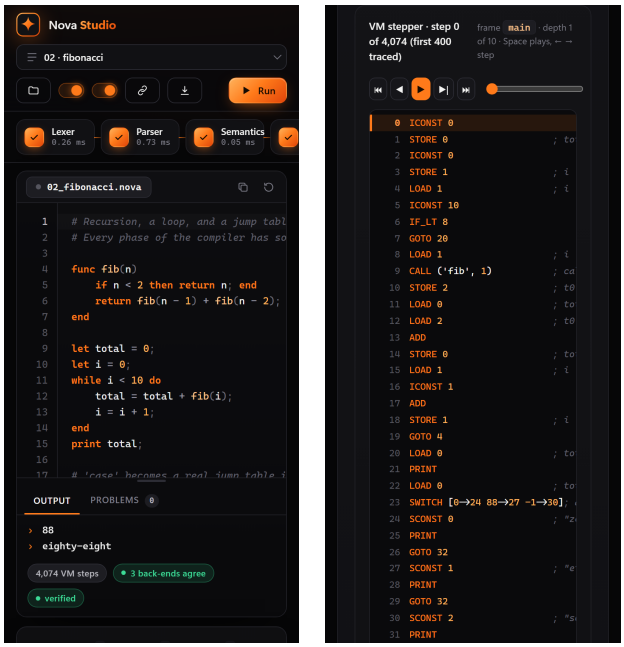
Fig. 34. The keyboard-shortcut panel. `Ctrl+Enter` runs with the full budget, `Alt+1…9` jump to a phase, and the space bar and arrow keys drive whichever player has focus.

over 31 non-terminals, and four conflicts remain. All four are common-prefix conflicts (`IDENT` followed by an assignment or a call, `if` with or without `else`, `case` with or without `otherwise`, and a variable or a call in an expression) that left factoring removes—the same situations in which the hand-written parser looks one token further.

TABLE VIII
SELF-TEST SUITE (34 TESTS)

Group	n	What is checked
M1	3	direct-method DFA; minimisation; maximal munch and keyword priority
M2	6	FIRST/FOLLOW; left recursion; ambiguity conflicts; SLR/CLR/LALR accept Nova; LALR agrees with recursive descent; table-cache regression
M3	3	type errors; shadowing; function-use flags (regression)
M4	2	short-circuit jumping code; jump tables and backpatching
M5	3	killed copies are not propagated; optimiser removes code; data-flow fixed points
M6	3	register and stack machines agree; optimised and unoptimised code agree; activation records
M7	3	ϕ placement; ϕ at an entry loop header (regression); dependence verdicts
M8	3	honest bytecode passes; every hostile program is stopped; no false rejection
E2E	6	expected outputs of ten programs; multi-phase diagnostics; HTML report; optimiser switch; deep nesting; budgets
WEB	2	JSON for every example; HTTP API with its 400/403/404/415 guards

The LR results reproduce the classical trade-off with real



(a) Overview (b) VM stepper

Fig. 35. The studio at a phone width of 390 pixels. The panes stack, the toolbar wraps, the pipeline rail scrolls sideways, and the instruments keep working.

TABLE IX
FRONT-END STATISTICS

Scanner	
Token patterns / keywords	41 / 19
Positions in the combined syntax tree	147
DFA states before / after minimisation	92 / 92
Symbol equivalence classes	37
Grammar	
Productions / terminals / non-terminals	58 / 39 / 20
LL(1) conflicts, as written	77
LL(1) conflicts after left-recursion elimination	4
Ambiguous expression grammar: shift/reduce conflicts	4

LR table	States	Entries	Conflicts	Build (ms)
SLR(1)	116	1,229	0	38.1
CLR(1)	478	4,790	0	1,771.6
LALR(1)	116	1,214	0	1,798.7

numbers. On this grammar, SLR(1), CLR(1) and LALR(1) all produce conflict-free tables, so the extra power of LR(1) look-aheads is not needed. CLR(1), however, splits the automaton into 478 states with 4,790 entries, whereas LALR(1) keeps SLR(1)'s 116 states and has 1,214 entries—75% fewer than CLR(1), and even 15 fewer than SLR(1), because merged LR(1) look-aheads are more precise than FOLLOW sets. This is the argument for LALR(1) in Yacc and Bison [6], [26], now measured rather than quoted. The automaton browser makes the merged look-aheads concrete: state 2 in Fig. 8 carries the look-ahead set $\{\$, \texttt{CASE}, \texttt{FUNC}, \ldots\}$ on every statement item, the union of the look-aheads of the LR(1) states that were merged into it.

Construction cost tells a second story. Because Nova obtains LALR(1) by merging CLR(1) states (2), it pays the full price of the canonical collection (1,798.7 ms against 38.1 ms for SLR(1), a factor of 47), which is exactly why production generators compute LALR look-aheads directly from the LR(0) automaton [25]. The cost is incurred once, when the server starts (3.8 s for all three tables on this machine), and is then cached.

The LALR(1) driver accepted every valid benchmark and rejected `broken`, in agreement with the recursive-descent parser; the bottom-up parses take 68–383 steps (Table X). On the ambiguous expression grammar, SLR(1) construction reports four shift/reduce conflicts, two in each of states 8 and 9 on + and $\star$ (Fig. 13), which is the expected signature of missing precedence and associativity. The parse player (Fig. 10) shows the same machinery from the driver's side: at step 32 the reduction `unary` $\rightarrow$ `factor` is dictated by the single kernel item of state 25, whose look-ahead set contains the look-ahead `;`.

C. End-to-End Compilation

Table X reports every phase for every benchmark. All ten valid programs compile, pass verification and run, and every property check succeeds (114 of 114). The intermediate code ranges from 6 to 38 quadruples per program (243 in total) with 58 backpatch operations, and SSA construction places 39 ϕ-functions across the suite. The front end recovers from errors in `broken` and stops before code generation, as designed. Compile times are 1.97–36.3 ms for nine of the ten valid programs; the tenth, `recursion`, takes 194.8 ms, of which 181.5 ms is execution—the program runs three times, and the VM re-renders the whole activation-record stack every time a new maximum depth is reached, which for a recursion 200 deep is 20,100 frame renderings. That cost is the price of the deepest-stack snapshot of Fig. 40, and it is discussed as a limitation in Section IV-J.

D. Optimisation Effectiveness

Table XI traces `main` of the `optimiser` benchmark pass by pass, and Fig. 19 shows its flow graph before and after. The first three passes change the code without changing its size: value numbering reuses one common subexpression (the shared node `n2` of Fig. 20) and folds one constant, and constant propagation substitutes one operand. LICM hoists the invariant product `width*height` into a preheader. Strength reduction then *adds* an instruction, introducing an incrementally updated induction variable in place of `i*8`, and dead-code elimination removes three instructions that had become dead, among them the definition of `n` after constant propagation and the never-read `unused`. The net effect is 20→18 instructions in `main`. The waterfall in Fig. 18 makes this non-monotone behaviour visible, where a single before-and-after count would hide it.

Table XII compares each program compiled with the optimiser switched off and on. The optimiser shrinks the intermediate code by up to 50%, the bytecode by up to 57% and the

TABLE X
COMPILATION RESULTS PER BENCHMARK (MEDIAN COMPILE TIME OVER 60 RUNS, BATTERY POWER)

Program	Tokens	LR steps	Symbols	TAC	Backp.	Blocks	IR	Loops	ϕ	Target	Spills	Bytecode	Checks	ms
`hello`	18	68	2	6	0	1	6→3	0	0	5	0	14→6	6/6	1.97
`fibonacci`	84	297	4	28	6	13	28→28	1	3	61	0	60→60	12/12	36.27
`gcd`	68	239	7	22	2	6	22→22	1	3	42	0	43→43	18/18	10.55
`primes`	103	363	6	31	11	21	31→31	2	9	68	0	71→71	12/12	28.57
`optimiser`	97	318	11	21	2	6	21→19	1	4	42	1	55→47	12/12	15.37
`parallel`	102	363	5	28	6	13	28→26	3	7	56	0	67→63	6/6	18.36
`types`	104	383	6	38	12	22	38→31	1	5	62	0	78→60	12/12	19.72
`broken`	33	22	3	*stopped after semantic analysis: 5 errors, 2 warnings*										0.90
`nested`	60	206	5	17	4	9	17→15	2	7	32	0	40→36	6/6	13.32
`strings`	88	314	6	29	11	19	29→25	0	1	50	0	57→46	12/12	12.04
`recursion`	62	227	4	23	4	9	23→23	0	0	47	0	50→50	18/18	194.81

TAC: three-address instructions; Backp.: backpatch operations; IR: instructions before→after optimisation; Target: register-machine instructions; Bytecode: unoptimised→optimised instructions; Checks: verifier properties proved; ms: median compile time including three executions and LLVM emission.

TABLE XI
PASS-BY-PASS OPTIMISATION OF `main` IN `optimiser`

Pass	Before	After	Δ	Effect
Local value numbering	20	20	0	1 CSE reused, 1 constant folded
Constant propagation	20	20	0	1 operand replaced
Copy propagation	20	20	0	no copy replaced
Loop-invariant motion	20	20	0	1 instruction hoisted
Strength reduction	20	21	+1	1 induction variable reduced
Dead-code elimination	21	18	−3	3 dead instructions removed
Unreachable code	18	18	0	every block reachable
Jump optimisation	18	18	0	no jump changed
Value numbering (2nd)	18	18	0	1 constant folded
Dead code (2nd)	18	18	0	nothing left to remove

TABLE XII
EFFECT OF THE OPTIMISER (OFF → ON)

	IR instructions			Bytecode			VM steps		
Program	off	on	%	off	on	%	off	on	%
`hello`	6	3	50	14	6	57	14	6	57
`fibonacci`	28	28	0	60	60	0	4,074	4,074	0
`gcd`	22	22	0	43	43	0	100	100	0
`primes`	31	31	0	71	71	0	1,610	1,610	0
`optimiser`	21	19	10	55	47	15	188	148	21
`parallel`	28	26	7	67	63	6	340	336	1
`types`	38	31	18	78	60	23	114	85	25
`nested`	17	15	12	40	36	10	264	260	2
`strings`	29	25	14	57	46	19	44	33	25
`recursion`	23	23	0	50	50	0	3,609	3,609	0

number of executed VM instructions by up to 57%. The largest reductions come from the smallest program, whose constant expressions fold away entirely; the largest on substantial programs are 23% (bytecode) and 25% (executed instructions) for `types`, and 19% and 25% for `strings`, where the `if`/`else` inside `greet` and the boolean expression leave dead temporaries and copies for the passes to remove. Four programs—`fibonacci`, `gcd`, `primes` and `recursion`—are unchanged: written idiomatically, they contain no redundancy for the classical machine-independent passes to remove, which is itself a useful lesson that optimisers mostly repair inefficient code. The `nested` benchmark shows the other extreme of the relation between static and dynamic savings: 10% fewer bytecode instructions but only 2% fewer executed, because the instructions removed lie outside the inner loop. Most importantly, every program prints exactly the same output with the optimiser on and off (Fig. 36).

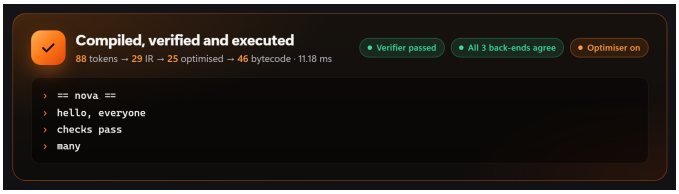
E. Data-Flow Convergence

Table XIII lists the number of round-robin passes that each analysis needs, counting the final pass that confirms the fixed point. No analysis needs more than four passes on any function, and the only function that needs four is `main` of `nested`, whose two nested loops give its flow graph a loop-connectedness of $d = 2$. This agrees with the bound of Kam and Ullman [31], under which rapid frameworks converge within $d + 2$ passes of a depth-first ordering: every other benchmark has $d \leq 1$ and converges within three. The visible convergence in Fig. 22, and the facts on the graph in Fig. 21, turn this theorem into something a student can watch.

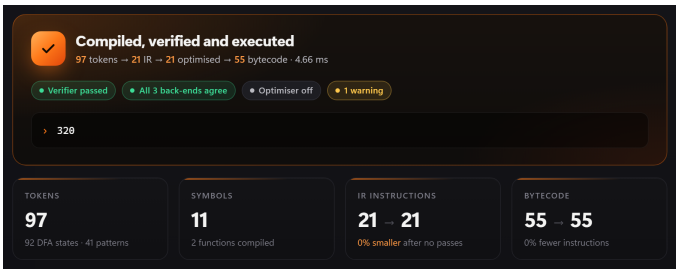
TABLE XIII
PASSES TO THE FIXED POINT PER FUNCTION (INCLUDING THE CONFIRMING PASS)

Program / function	RD	LV	AE	VB
`hello/main`	1	1	1	2
`fibonacci/fib`	2	2	2	2
`fibonacci/main`	2	3	2	1
`gcd/gcd`	2	3	2	1
`gcd/lcm`	2	2	1	2
`gcd/main`	1	1	1	1
`primes/isprime`	3	3	2	3
`primes/main`	3	3	2	1
`optimiser/measure`	2	2	1	1
`optimiser/main`	2	3	2	2
`parallel/main`	2	3	2	1
`types/classify`	2	2	1	1
`types/main`	2	3	2	2
`nested/main`	3	4	2	1
`strings/greet`	2	2	1	1
`strings/main`	2	2	2	2
`recursion/depth`	2	2	2	2
`recursion/fact`	2	2	2	2
`recursion/main`	1	1	1	1

RD: reaching definitions; LV: live variables; AE: available expressions; VB: very busy expressions.



(a) `10_strings.nova`: string concatenation, a boolean, an `if`/`else` inside a function and a `case` with `otherwise`, all verified and agreed by the three back-ends.



(b) `05_optimiser.nova` with the optimiser switched off: 21 IR and 55 bytecode instructions instead of 19 and 47, the same output `320`, and the switch's state shown in the summary.

Fig. 36. Two overview summaries. The optimiser switch recompiles the program through the unoptimised path so that its effect, and the invariance of the output, can be seen directly.

TABLE XIV
EXECUTION ON THE THREE BACK-ENDS

Program	Stack VM steps unopt.	Stack VM steps opt.	Register steps	Max frames	Max stack	Agree
`hello`	14	6	5	1	1	✓
`fibonacci`	4,074	4,074	3,819	10	2	✓
`gcd`	100	100	91	3	2	✓
`primes`	1,610	1,610	1,485	2	2	✓
`optimiser`	188	148	119	2	2	✓
`parallel`	340	336	290	1	2	✓
`types`	114	85	87	2	2	✓
`nested`	264	260	223	1	2	✓
`strings`	44	33	37	2	2	✓
`recursion`	3,609	3,609	3,188	202	2	✓

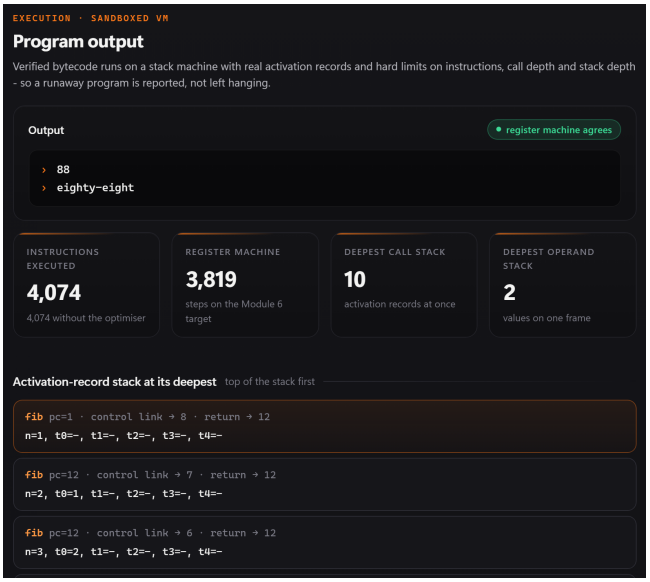


Fig. 37. Execution view for the Fibonacci benchmark: the program output (confirmed by the register machine), instruction counts on both targets, and the activation-record stack at its deepest, top first. Each frame shows its function, program counter, control link, return address and the current values of its locals.

F. Code Generation and Back-End Agreement

Table XIV reports the three executions of each program. On every benchmark the unoptimised stack VM, the optimised stack VM and the register machine print identical output, which is the differential check of correctness [55]. A single spill occurs in the whole suite (in `optimiser`), so four registers suffice for these programs; the timeline of Fig. 24 shows why, since at most four values are live at once and constants occupy a register for a single instruction. The register machine executes 6–25% fewer instructions than the optimised stack VM on nine of the ten programs, because it keeps values in registers instead of pushing and popping them. The exception is `types` (87 against 85 instructions), where the code generator stores live values back at block boundaries and the stack code needs no such stores. The recursive `fibonacci` reaches a depth of ten activation records, which Fig. 37 shows frame by frame, and `recursion` reaches 202 (Fig. 40).

G. SSA and Parallelism

The suite contains eleven natural loops. The dependence analyser classifies two as vectorisable (in `isprime` and the first loop of `parallel`), six as reductions (the accumulators `total`, `sum`, `acc` and `b`, and both loops of `nested`, which accumulate `total`) and three as sequential (`gcd`'s Euclidean step, the recurrence on `c` in `parallel` and the countdown on `n` in `types`). Each verdict matches a manual analysis of the loop, and Fig. 28 shows how three syntactically similar loops in `parallel` receive three different verdicts—the distinction a vectoriser must make before transforming anything [46]. For the nested pair (Fig. 29) the analyser additionally reports the nesting depth and recommends interchange and tiling, the two transformations Kennedy and McKinley [48] use to improve locality. The 39 ϕ-functions follow the iterated dominance frontiers exactly; the dominator tree of `isprime` (Fig. 27) shows the frontier $\{$`B4`$\}$ on every block that can re-enter the loop header, which is where its nine ϕ-functions go. For `gcd`, whose loop header is its entry block, Fig. 26 shows the ϕ-functions that a defect previously suppressed (Section IV-K).

H. Verification and Security

All 114 property checks on the honest programs pass (Fig. 41), and verification is cheap: 0.054–0.480 ms per pro-

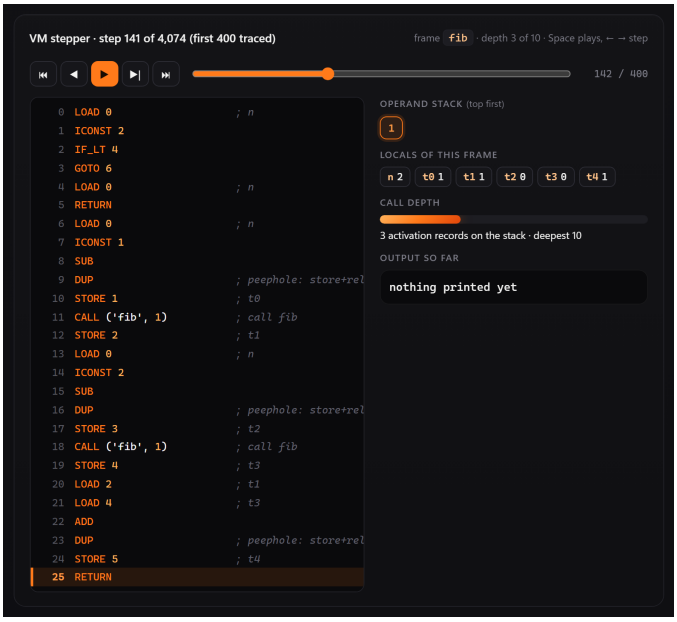


Fig. 38. The VM stepper at step 141 of the Fibonacci run, three frames deep inside `fib`: the program counter sits on `RETURN`, the operand stack holds the value `1` about to be returned, the locals show `n=2` and the two partial results `t0` and `t4`, and nothing has been printed yet.

TABLE XV
HOSTILE BYTECODE SUITE

#	Attack	Violates	Stopped by
1	Branch outside the code	P3	verifier
2	Operand-stack underflow	P1/P2	verifier
3	Inconsistent depth at a merge	P1	verifier
4	Type confusion (string × int)	P2	verifier
5	Read of an uninitialised local	P4	verifier
6	Slot outside the activation record	P5	verifier
7	Non-terminating loop	budget	sandbox
8	Unbounded recursion	frame limit	sandbox

gram, under 3% of compile time. Table XV reports the hostile suite, which reuses the same verifier and VM with tighter run-time limits of 200,000 instructions and 64 frames. All eight attacks are stopped. Six are memory- or type-safety violations that the verifier rejects before execution, each named by the property it violates (Fig. 42). The last two are well typed and therefore pass verification; they are denials of service, which no static check can decide in general, and the sandbox stops them at run time (Fig. 43). This is the division of labour between validation and resource limits that WebAssembly and eBPF also adopt [11], [12].

I. Diagnostics and Robustness

On `broken`, the front end reports seven diagnostics in one run: one lexical error, two syntax errors, two semantic errors and two warnings, each with its line and column and a hint (Fig. 44). Panic-mode recovery and the poisoned error type prevent cascades: every error in the program is reported once, and no spurious errors are added.

The studio was also probed with inputs designed to break it. A program with 3,000 nested parentheses previously crashed the compiler with a Python recursion error; it now yields a syntax diagnostic in 20.7 ms. Unbounded recursion is stopped at 256 frames. A non-terminating loop is stopped after 1.09 s under the live budget, against 8.75 s under the full one, and the studio tells the user which budget applied (Fig. 45). This eight-fold faster feedback is what makes live recompilation safe.

J. Performance, Interactivity and Scaling

Fig. 46 breaks the median compile time of each program into phases. A complete compilation—all eight phases, three executions and LLVM emission—takes 1.97–36.3 ms for nine of the ten valid programs and 0.90 ms for `broken`. Work in modules 5–7 and execution dominate; the front end costs at most 5.5 ms. For `fibonacci`, execution accounts for 77% of the time, because the program runs three times (11,967 VM instructions in total) and the optimised run records a trace with the locals of every step. Measured from a separate process, as a browser uses it, an HTTP compile request takes 7.4–49.2 ms end to end for these programs and returns a 26–174 KB JSON document; a bare health check takes 0.74 ms, so building, serialising and transferring the full glass-box view adds 5–13 ms to the compile itself. Including the 360 ms debounce, an updated view arrives about 0.4 s after the last keystroke; the debounce, not the compiler, sets the latency, and the phases themselves stay inside the 0.1 s limit for instantaneous response [67] for every benchmark except the deep recursion. The page itself is 7.7 KB of HTML, 52.6 KB of CSS and 116.8 KB of JavaScript, plus the 160 KB grammar document fetched once per session.

Scaling. Table XVI and Fig. 47 report the synthetic series. Compile time grows faster than the program: doubling the program from 56 to 112 lines costs 2.8×, from 112 to 224 lines 4.8×, and from 224 to 448 lines 7.8×, so the growth lies between quadratic and cubic. The front end stays close to linear (16 ms of lexing and 54 ms of parsing for 2,112 tokens and 7,299 parse steps at the largest size), and execution and verification are small. Almost all of the growth is in modules 5–7: 4,935 of the 5,297 ms at 64 loops. The reasons are the glass box's own bookkeeping, which was never designed for large inputs: dominators are computed by iterative intersection of whole sets, which is $O(B^2)$ in the number of blocks B; each of the four data-flow analyses copies every block's sets on every round and keeps a snapshot of all of them per round; and the optimiser records the complete code listing after each of its ten passes. These are the costs of keeping everything, and they bound the size of program for which the studio is instantaneous: on this throttled machine, programs of about 60 lines compile within 0.1 s and programs of about 110 lines within 0.15 s, while a 448-line program takes 5.3 s. Section V lists the standard remedies. The JSON document grows with

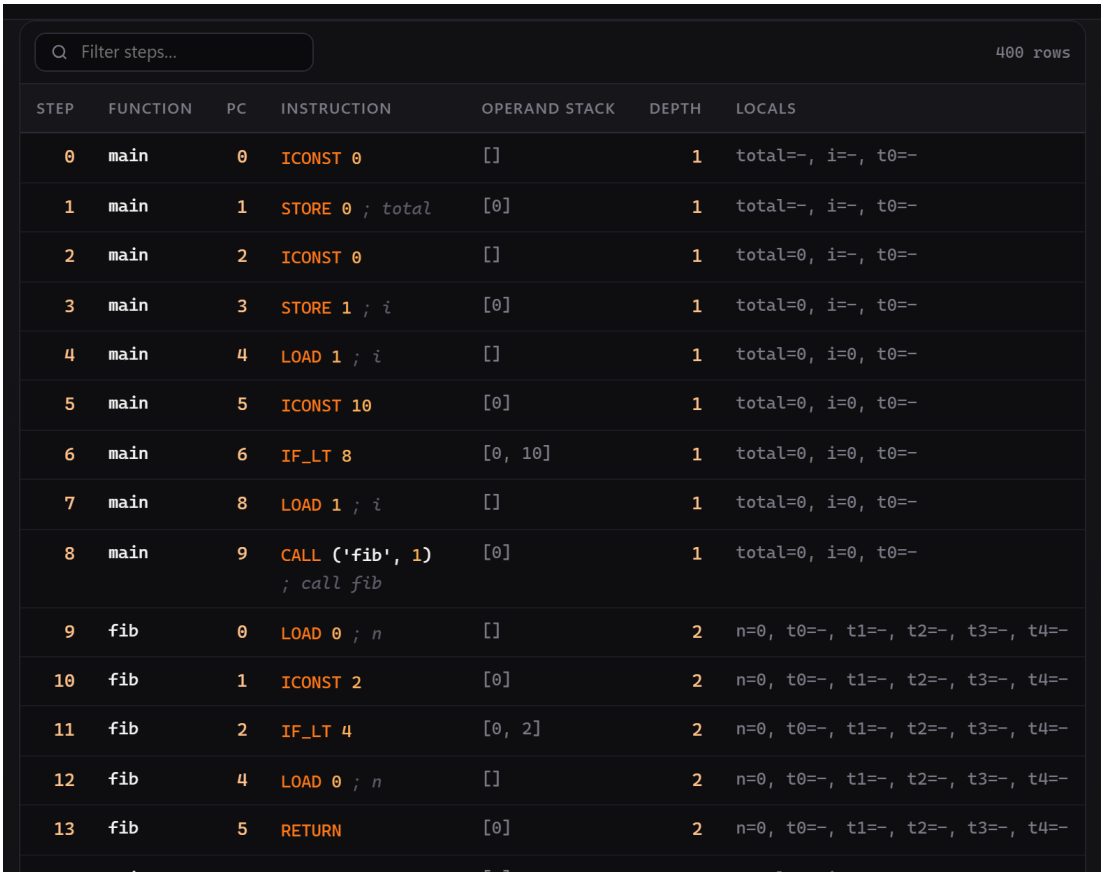


Fig. 39. The first fourteen rows of the execution trace of the Fibonacci program, with the operand stack, the call depth and the locals of the current frame at every step. Step 8 is the first `CALL`; step 9 begins inside `fib` with `n=0`.

TABLE XVI
SCALING WITH PROGRAM SIZE (SYNTHETIC PROGRAMS OF N LOOPS)

N	Lines	Tokens	LR steps	Blocks	IR	ms	JSON KB
1	7	33	117	5	10	7.8	57
2	14	66	231	9	19	10.6	98
4	28	132	459	17	37	22.8	194
8	56	264	915	33	73	51.1	378
16	112	528	1,827	65	145	140.8	757
32	224	1,056	3,651	129	289	678.4	1,598
64	448	2,112	7,299	257	577	5,296.7	3,705

the program as well—57 KB to 3.7 MB across the series—but remains bounded by the row caps.

K. Defects Exposed by the Glass Box

Building, measuring and screenshotting every phase exposed six functional defects (Table XVII); all are fixed, and five are covered by new regression tests. Two of them are telling. The SSA defect (D6) left `gcd` with zero ϕ-functions—a count that was plainly wrong once it appeared in a measurement table—because the dominance-frontier routine admitted only blocks with two or more recorded predecessors as join points and ignored the implicit edge into the entry block. The symbol-table defect (D5) became visible in a screenshot taken for this report, which showed a function that is called three times as unused. Neither defect affected program output, so output-only testing could never have found them. Both were found because the artefacts were on screen. This is the strongest evidence in this study for the glass-box approach: a tool that shows its working also exposes its own mistakes.

L. Educational Value and Other Front Ends

Because one `CompileResult` drives every view, the same artefacts appear in the desktop viewer and the self-contained HTML report (Figs. 49 and 50), which can be e-mailed or printed and requires no server. Taken together, the studio covers every module of the syllabus (Table II and Fig. 33) with an inspectable artefact of a real compilation rather than a static diagram: a student can change a program and immediately see how the token stream, the parse, the symbol table, the flow graph, the data-flow solution, the SSA form and the verifier's proof change with it; and, following the engagement findings of the visualisation literature [13], [14], the student can also *operate* each instrument—choose the token, the state, the block, the analysis or the step—rather than only look at it.

TABLE XVII
DEFECTS EXPOSED WHILE BUILDING AND EVALUATING NOVA STUDIO

ID	Component	Symptom	Root cause	Fix	Test
D1	LR table cache	CLR(1) table silently missing from every later report	cache keyed on the grammar alone, and CLR built only on a cold, "deep" request	build CLR on a cache hit when it is needed	M2
D2	Front end	crash (Python recursion error) on 3,000 nested parentheses	recursive descent and tree walks exceed the interpreter's recursion limit	convert to a syntax diagnostic at the pipeline boundary	E2E
D3	Desktop viewer	"optimiser on" check box had no effect	the pipeline offered no switch to wire it to	`optimise` flag through to bytecode and peephole	E2E
D4	Desktop viewer	internal-error handler itself crashed	it wrote text into a table widget	report the error as a diagnostic row	manual
D5	Semantic analysis	called functions listed as unused	call checking never marked the callee's symbol	mark the callee on every call	M3
D6	SSA	no ϕ-functions when a function begins with a loop (wrong SSA for `gcd`)	join test ignored the implicit edge into the entry block	count that edge; render an `entry` operand	M7

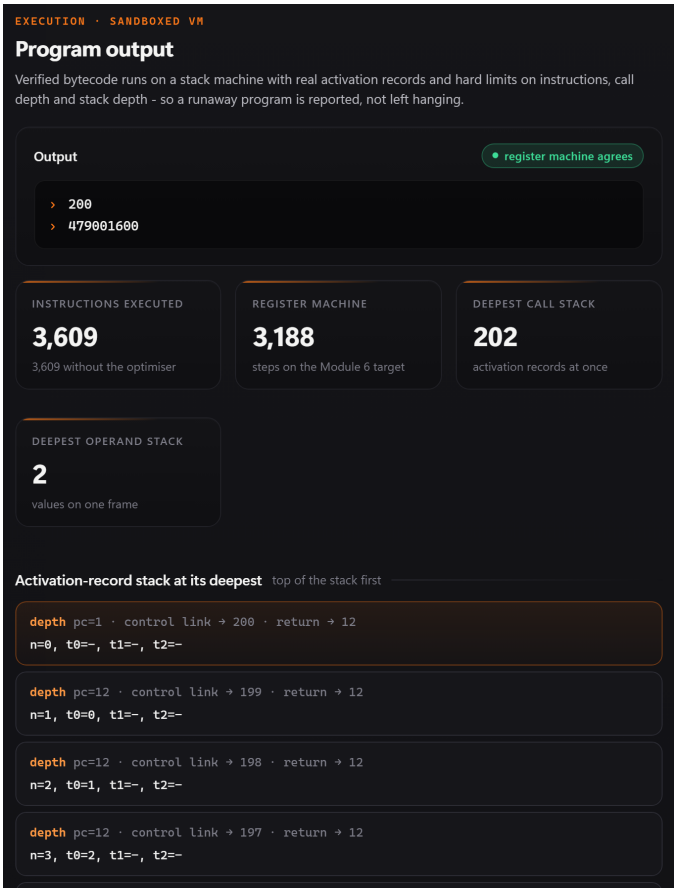


Fig. 40. `11_deep_recursion.nova`: 202 activation records at the deepest point—`main`, then 200 frames of `depth` and the frame that returns 0—each with its control link to the frame below, inside the sandbox's limit of 256. Changing `200` to `300` makes the sandbox stop the program instead.

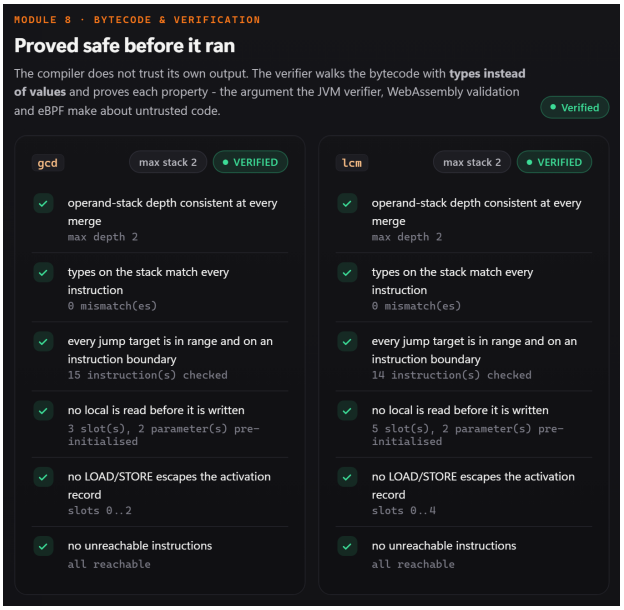


Fig. 41. Verifier view for `gcd` and `lcm`: six properties proved for each function before execution, with the evidence for each, such as the maximum stack depth, the number of jump targets checked and the slots that are pre-initialised.

M. Limitations and Threats to Validity

Nova deliberately trades breadth for transparency, which imposes seven limitations:

- The language has no arrays, records or heap, so the dependence analysis reasons about scalars only and cannot compute dependence distances.
- Register allocation is local to basic blocks.
- LALR(1) is built from the canonical collection rather than by look-ahead propagation.
- The abstract domain of the verifier is coarse (`int`, `str`, `any`).
- Execution is interpreted in Python, and the deepest-stack snapshot costs $O(d^2)$ frame renderings for a recursion of depth d.
- The bookkeeping of modules 5–7 grows between quadratically and cubically with program size, which limits instantaneous feedback to programs of roughly a hundred lines.
- Nova Studio is a single-user local tool.

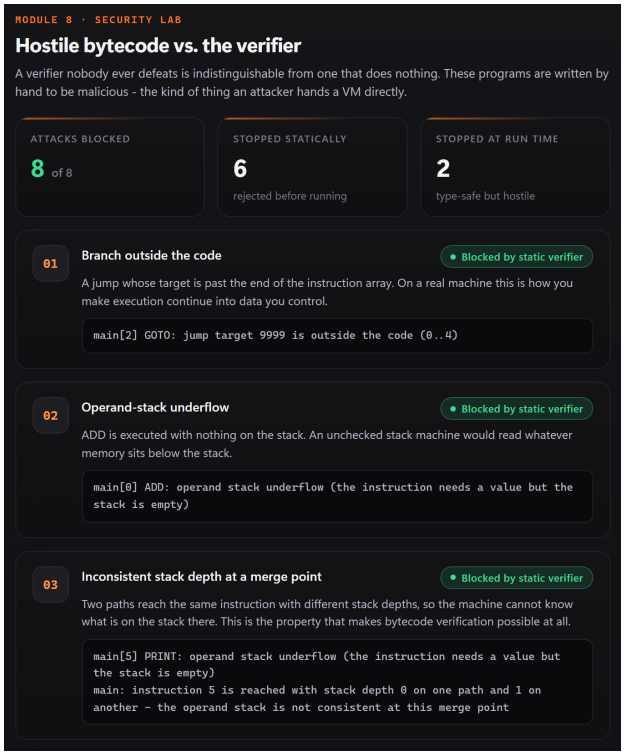


Fig. 42. Security lab: eight hostile programs handed directly to the VM, eight blocked—six statically, two at run time. The first three are shown, each with the property violated and the verifier's exact message.

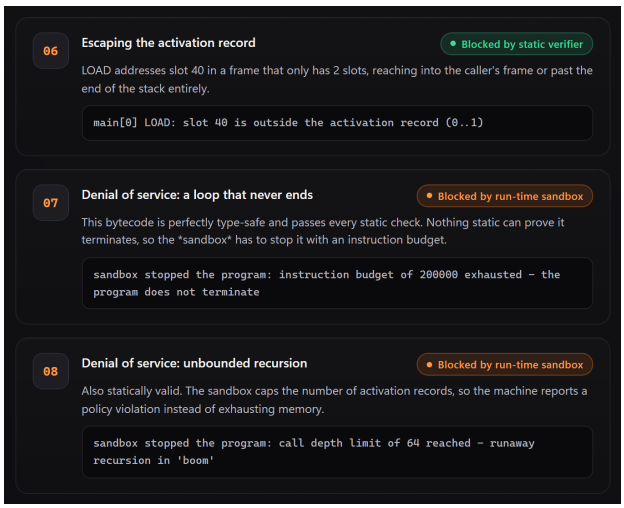


Fig. 43. The last three attacks. The frame-escaping `LOAD` is rejected statically; the non-terminating loop and the unbounded recursion pass every static check and are stopped by the sandbox's instruction budget and frame limit.

Three threats bear on the evaluation itself:

- *External validity.* The benchmark suite is small and was written alongside the compiler, so the optimisation figures describe these programs rather than real-world code, and the scaling series is made of copies of one loop.
- *Internal validity.* Timings come from one machine on battery power under normal desktop load; we mitigated noise with medians, CPU-time cross-checks, a separate server process and a recorded power state, and the relative comparisons were stable across repeated sessions, including a mains-powered one that ran about $3.5\times$ faster.
- *Construct validity.* The educational benefit is argued from coverage, engagement design and the visualisation literature, not yet measured with learners.

V. Conclusions and Future Work

This paper presented Nova, a glass-box compiler that takes an original language from regular expressions to verified, sandboxed execution while retaining and displaying every artefact on the way, and Nova Studio, which turns those artefacts into instruments a learner can operate. The evaluation shows six results:

- The classical claims of compiler theory hold when measured. LALR(1) keeps SLR(1)'s 116 states with 75% fewer entries than CLR(1), and every data-flow analysis converges within the $d + 2$ passes that theory predicts, including the doubly nested loop that needs four.
- Optimisation never changed a program's output, and the three independent back-ends agreed on every one of ten programs.
- The verifier and the sandbox stopped all eight hostile programs, six statically and two at run time.
- Compilation is fast enough for keystroke-level feedback on programs of the size used in a course—under 40 ms for nine of ten benchmarks even on a throttled laptop—and a scaling study locates precisely where the glass box's bookkeeping grows superlinearly.
- Making the internals visible exposed six latent defects that output-based testing had missed.
- The approach needs nothing beyond a standard Python installation and a browser.

Future work follows directly from the limitations:

1) replacing the iterative dominator computation by the Cooper–Harvey–Kennedy algorithm [39], solving the data-flow analyses with worklists over bit-vectors, and recording per-pass code as diffs, so that the middle end scales linearly in practice and the deepest-stack snapshot is taken once;
2) adding arrays and subscripted references, so that GCD and Banerjee dependence tests can compute distance vectors and drive loop distribution, fusion and interchange [46], [48];
3) constructing LALR(1) by DeRemer–Pennello lookahead propagation [25] and comparing it with the current core-merging method in the automaton browser;
4) global register allocation by graph colouring and by linear scan [41], [42], visualised on the same timeline as the current local allocator;
5) SSA-based optimisation, such as sparse conditional constant propagation [40] and global value numbering, replacing the iterative passes;
6) a mechanised proof that the verifier is sound;

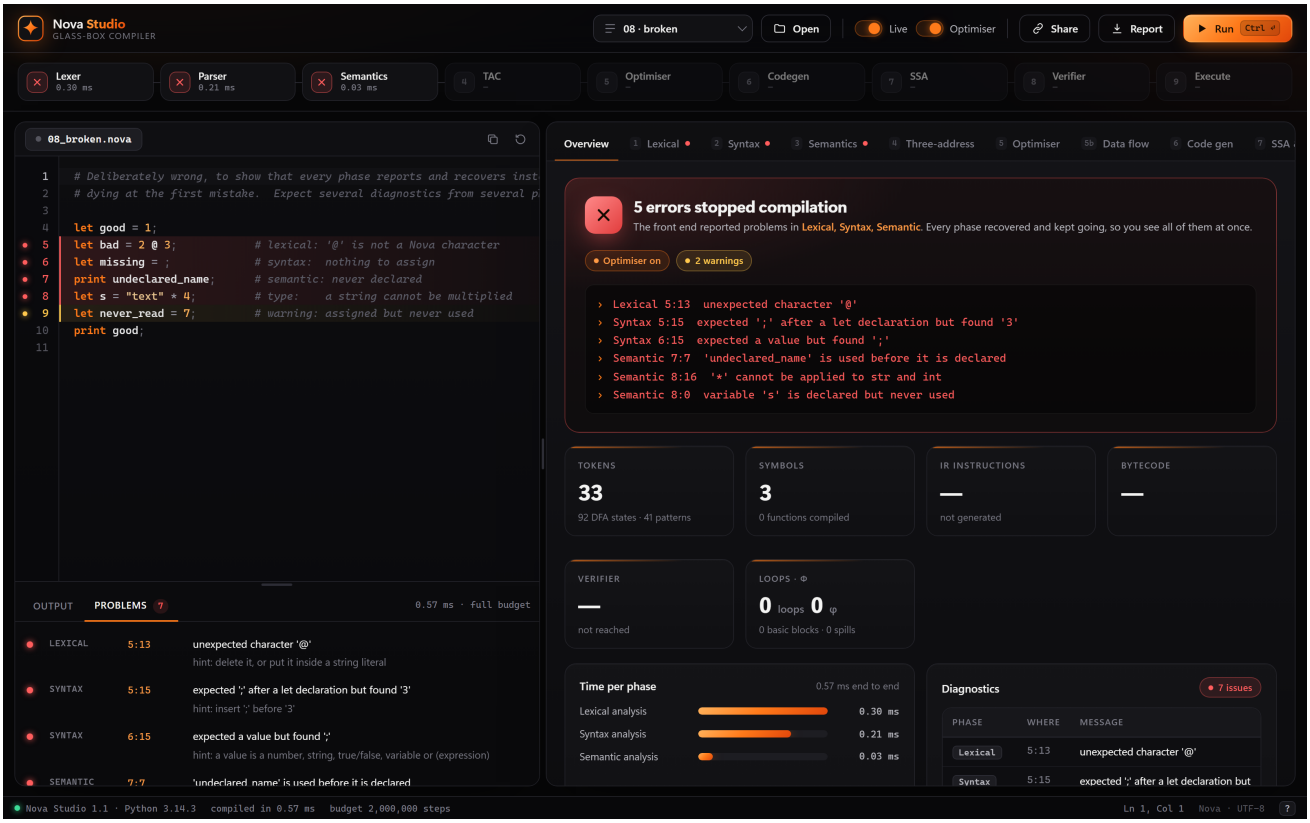


Fig. 44. Error recovery in Nova Studio on `08_broken.nova`. The rail shows the lexer, parser and semantic analyser failing and later stages skipped. The gutter marks the four lines with errors in red and the line with only a warning in amber. The problems panel lists all seven diagnostics with hints, and each entry jumps to its line when clicked.

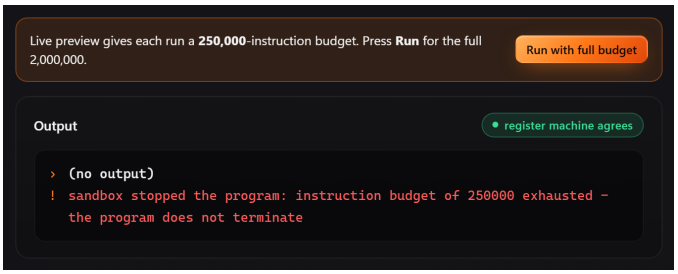


Fig. 45. A loop typed without its exit condition, stopped by the 250,000-instruction live budget. The studio explains the budget and offers a one-click run with the full 2,000,000-instruction budget.

7) a native back-end through LLVM or WebAssembly;
8) a controlled classroom study measuring whether the glass-box studio, and in particular its players, improve learning outcomes in a compiler course.

Acknowledgment

The authors thank [Faculty Name], [Department], [Institution], for guidance throughout the Compiler Design course project.

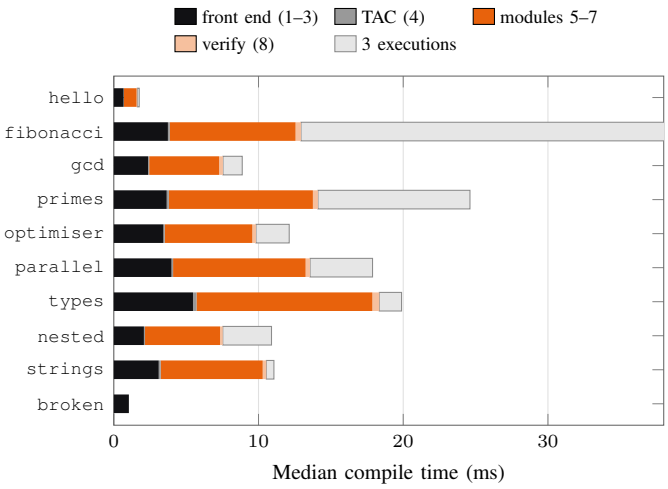


Fig. 46. Median compile time per benchmark (60 runs, battery power), split into front end, three-address code, modules 5–7 (blocks, data flow, optimisation, register code generation, SSA and dependence analysis, bytecode generation), verification and the three executions. `recursion` (194.8 ms, of which 181.5 ms is execution) is omitted from the axis. The remaining 0.1–1.3 ms per program, not drawn, is LLVM IR emission and bookkeeping.

References

[1] A. V. Aho, M. S. Lam, R. Sethi, and J. D. Ullman, *Compilers: Principles, Techniques, and Tools*, 2nd ed. Boston, MA, USA: Addison-Wesley,

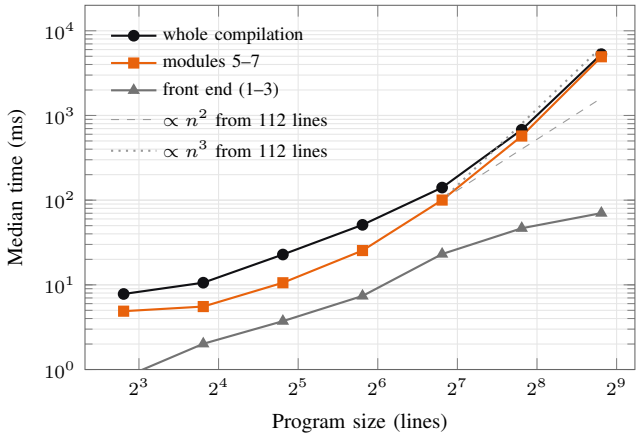


Fig. 47. Compile time against program size on a log–log scale for the synthetic series. The front end grows roughly linearly; modules 5–7 grow between quadratically and cubically, and dominate from about 100 lines upwards.

MEASUREMENTS

Numbers, not claims

Everything below was produced by this compilation.

Filter measurements... 30 rows

MODULE	MEASUREMENT	VALUE
Module 1	Token patterns (extended regular expressions)	41
Module 1	Positions in the combined syntax tree	147
Module 1	DFA states before minimisation	92
Module 1	DFA states after minimisation	92
Module 1	Symbol equivalence classes (DFA columns)	37
Module 1	Tokens produced	84
Module 2	Grammar productions	58
Module 2	Terminals / non-terminals	39 / 20
Module 2	SLR states / table entries / conflicts	116 / 1229 / 0
Module 2	CLR states / table entries / conflicts	478 / 4790 / 0
Module 2	LALR states / table entries / conflicts	116 / 1214 / 0
Module 2	Bottom-up parse of this program	accepted
Module 2	LR parser steps	297
Module 3	Symbol-table entries	4

Fig. 48. The metrics view: every measurement of the current compilation, produced by the compiler itself, from DFA states and parse-table sizes to verification results and VM step counts.

2006.

[2] A. W. Appel and J. Palsberg, *Modern Compiler Implementation in Java*, 2nd ed. Cambridge, U.K.: Cambridge Univ. Press, 2002.

[3] K. D. Cooper and L. Torczon, *Engineering a Compiler*, 2nd ed. San Francisco, CA, USA: Morgan Kaufmann, 2011.

[4] Free Software Foundation, "GCC developer options," in *Using the GNU Compiler Collection*. [Online]. Available: https://gcc.gnu.org/onlinedocs/gcc/Developer-Options.html

[5] M. E. Lesk and E. Schmidt, "Lex—A lexical analyzer generator," Bell Laboratories, Murray Hill, NJ, USA, Comput. Sci. Tech. Rep. 39, 1975.

[6] S. C. Johnson, "Yacc: Yet another compiler-compiler," Bell Laboratories, Murray Hill, NJ, USA, Comput. Sci. Tech. Rep. 32, 1975.

[7] T. J. Parr and R. W. Quong, "ANTLR: A predicated-LL(k) parser generator," *Softw.: Pract. Exper.*, vol. 25, no. 7, pp. 789–810, Jul. 1995.

[8] X. Yang, Y. Chen, E. Eide, and J. Regehr, "Finding and understanding bugs in C compilers," in *Proc. 32nd ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2011, pp. 283–294.

[9] T. Lindholm, F. Yellin, G. Bracha, and A. Buckley, *The Java Virtual Machine Specification*, Java SE 8 ed. Upper Saddle River, NJ, USA: Addison-Wesley, 2014.

[10] X. Leroy, "Java bytecode verification: Algorithms and formalizations," *J. Autom. Reasoning*, vol. 30, no. 3–4, pp. 235–269, 2003.

[11] A. Haas *et al.*, "Bringing the web up to speed with WebAssembly," in *Proc. 38th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2017, pp. 185–200.

[12] E. Gershuni *et al.*, "Simple and precise static analysis of untrusted Linux kernel extensions," in *Proc. 40th ACM SIGPLAN Conf. Program. Lang. Des. Implement. (PLDI)*, 2019, pp. 1069–1084.

[13] C. D. Hundhausen, S. A. Douglas, and J. T. Stasko, "A meta-study of algorithm visualization effectiveness," *J. Vis. Lang. Comput.*, vol. 13, no. 3, pp. 259–290, Jun. 2002.

[14] J. Sorva, V. Karavirta, and L. Malmi, "A review of generic program visualization systems for introductory programming education," *ACM Trans. Comput. Educ.*, vol. 13, no. 4, art. 15, Nov. 2013.

[15] S. S. Muchnick, *Advanced Compiler Design and Implementation*. San Francisco, CA, USA: Morgan Kaufmann, 1997.

[16] K. Thompson, "Programming techniques: Regular expression search algorithm," *Commun. ACM*, vol. 11, no. 6, pp. 419–422, Jun. 1968.

[17] R. McNaughton and H. Yamada, "Regular expressions and state graphs for automata," *IRE Trans. Electron. Comput.*, vol. EC-9, no. 1, pp. 39–47, Mar. 1960.

[18] V. M. Glushkov, "The abstract theory of automata," *Russian Math. Surveys*, vol. 16, no. 5, pp. 1–53, 1961.

[19] G. Berry and R. Sethi, "From regular expressions to deterministic automata," *Theor. Comput. Sci.*, vol. 48, pp. 117–126, 1986.

[20] E. F. Moore, "Gedanken-experiments on sequential machines," in *Automata Studies*, C. E. Shannon and J. McCarthy, Eds. Princeton, NJ, USA: Princeton Univ. Press, 1956, pp. 129–153.

[21] J. Hopcroft, "An $n \log n$ algorithm for minimizing states in a finite automaton," in *Theory of Machines and Computations*, Z. Kohavi and A. Paz, Eds. New York, NY, USA: Academic Press, 1971, pp. 189–196.

[22] D. E. Knuth, "On the translation of languages from left to right," *Inf. Control*, vol. 8, no. 6, pp. 607–639, Dec. 1965.

[23] F. L. DeRemer, "Simple LR(k) grammars," *Commun. ACM*, vol. 14, no. 7, pp. 453–460, Jul. 1971.

[24] F. L. DeRemer, "Practical translators for LR(k) languages," Ph.D. dissertation, Dept. Elect. Eng., MIT, Cambridge, MA, USA, 1969.

[25] F. DeRemer and T. Pennello, "Efficient computation of LALR(1) look-ahead sets," *ACM Trans. Program. Lang. Syst.*, vol. 4, no. 4, pp. 615–649, Oct. 1982.

[26] Free Software Foundation, "GNU Bison manual: Understanding your parser." [Online]. Available: https://www.gnu.org/software/bison/manual/

[27] D. E. Knuth, "Semantics of context-free languages," *Math. Syst. Theory*, vol. 2, no. 2, pp. 127–145, 1968.

[28] P. M. Lewis, D. J. Rosenkrantz, and R. E. Stearns, "Attributed translations," *J. Comput. Syst. Sci.*, vol. 9, no. 3, pp. 279–307, Dec. 1974.

[29] F. E. Allen, "Control flow analysis," *SIGPLAN Not.*, vol. 5, no. 7, pp. 1–19, Jul. 1970.

[30] G. A. Kildall, "A unified approach to global program optimization," in *Proc. 1st ACM SIGACT-SIGPLAN Symp. Princ. Program. Lang. (POPL)*, 1973, pp. 194–206.

[31] J. B. Kam and J. D. Ullman, "Global data flow analysis and iterative algorithms," *J. ACM*, vol. 23, no. 1, pp. 158–171, Jan. 1976.

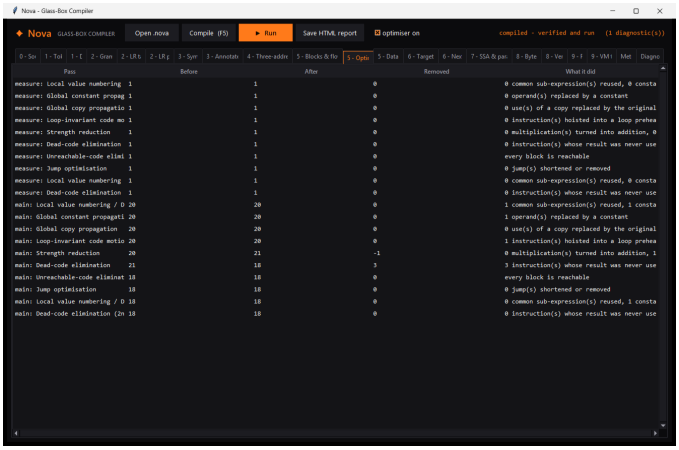
[32] J. B. Kam and J. D. Ullman, "Monotone data flow analysis frameworks," *Acta Inform.*, vol. 7, no. 3, pp. 305–317, 1977.

[33] J. Cocke, "Global common subexpression elimination," *SIGPLAN Not.*, vol. 5, no. 7, pp. 20–24, Jul. 1970.

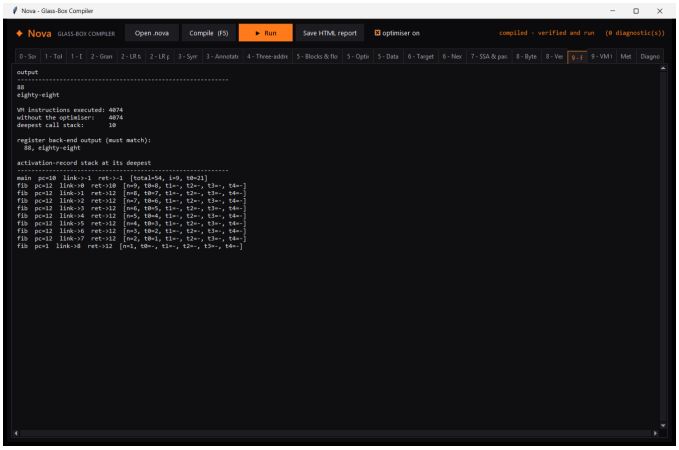
[34] B. Alpern, M. N. Wegman, and F. K. Zadeck, "Detecting equality of variables in programs," in *Proc. 15th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 1988, pp. 1–11.

[35] F. E. Allen, J. Cocke, and K. Kennedy, "Reduction of operator strength," in *Program Flow Analysis: Theory and Applications*, S. S. Muchnick and N. D. Jones, Eds. Englewood Cliffs, NJ, USA: Prentice-Hall, 1981, pp. 79–101.

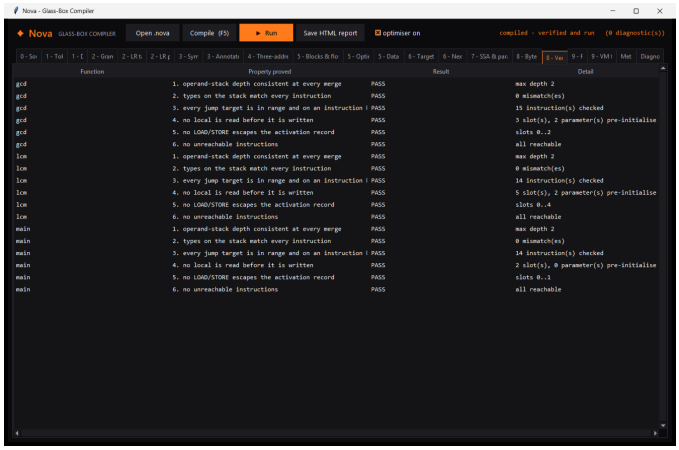
[36] W. M. McKeeman, "Peephole optimization," *Commun. ACM*, vol. 8, no. 7, pp. 443–444, Jul. 1965.



(a) Optimiser tab: the pass records of both functions of `05_optimiser.nova`



(b) Run tab: output, step counts and the activation-record stack of the Fibonacci program



(c) Verifier tab: the six properties proved for each of the three functions of `03_gcd.nova`

Fig. 49. The desktop viewer (Tkinter), with one tab per phase, in the same black-and-orange visual identity as the studio. It needs no browser and no server, only the Python installer's own Tkinter.

[37] R. Cytron, J. Ferrante, B. K. Rosen, M. N. Wegman, and F. K. Zadeck, "Efficiently computing static single assignment form and the control dependence graph," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 4, pp. 451–490, Oct. 1991.

[38] T. Lengauer and R. E. Tarjan, "A fast algorithm for finding dominators in a flowgraph," *ACM Trans. Program. Lang. Syst.*, vol. 1, no. 1, pp. 121–141, Jul. 1979.

[39] K. D. Cooper, T. J. Harvey, and K. Kennedy, "A simple, fast dominance algorithm," Rice Univ., Houston, TX, USA, Tech. Rep. TR-06-33870, 2001.

[40] M. N. Wegman and F. K. Zadeck, "Constant propagation with conditional branches," *ACM Trans. Program. Lang. Syst.*, vol. 13, no. 2, pp. 181–210, Apr. 1991.

[41] G. J. Chaitin, "Register allocation & spilling via graph coloring," in *Proc. SIGPLAN Symp. Compiler Construction*, 1982, pp. 98–105.

[42] M. Poletto and V. Sarkar, "Linear scan register allocation," *ACM Trans. Program. Lang. Syst.*, vol. 21, no. 5, pp. 895–913, Sep. 1999.

[43] L. A. Belady, "A study of replacement algorithms for a virtual-storage computer," *IBM Syst. J.*, vol. 5, no. 2, pp. 78–101, 1966.

[44] C. Lattner and V. Adve, "LLVM: A compilation framework for lifelong program analysis & transformation," in *Proc. Int. Symp. Code Gener. Optim. (CGO)*, 2004, pp. 75–86.

[45] R. Allen and K. Kennedy, "Automatic translation of FORTRAN programs to vector form," *ACM Trans. Program. Lang. Syst.*, vol. 9, no. 4, pp. 491–542, Oct. 1987.

[46] R. Allen and K. Kennedy, *Optimizing Compilers for Modern Architectures: A Dependence-Based Approach*. San Francisco, CA, USA: Morgan Kaufmann, 2001.

[47] M. Wolfe, *High Performance Compilers for Parallel Computing*. Redwood City, CA, USA: Addison-Wesley, 1996.

[48] K. Kennedy and K. S. McKinley, "Maximizing loop parallelism and improving data locality via loop fusion and distribution," in *Languages and Compilers for Parallel Computing* (Lecture Notes in Computer Science, vol. 768). Berlin, Germany: Springer, 1994, pp. 301–320.

[49] OpenMP Architecture Review Board, "OpenMP application programming interface, version 5.2," Nov. 2021. [Online]. Available: https://www.openmp.org/specifications/

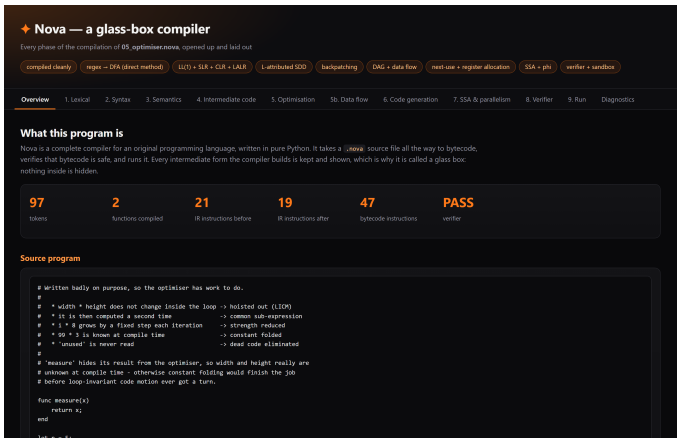
[50] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM SIGACT-SIGPLAN Symp. Princ. Program. Lang. (POPL)*, 1977, pp. 238–252.

[51] G. C. Necula, "Proof-carrying code," in *Proc. 24th ACM SIGPLAN-SIGACT Symp. Princ. Program. Lang. (POPL)*, 1997, pp. 106–119.

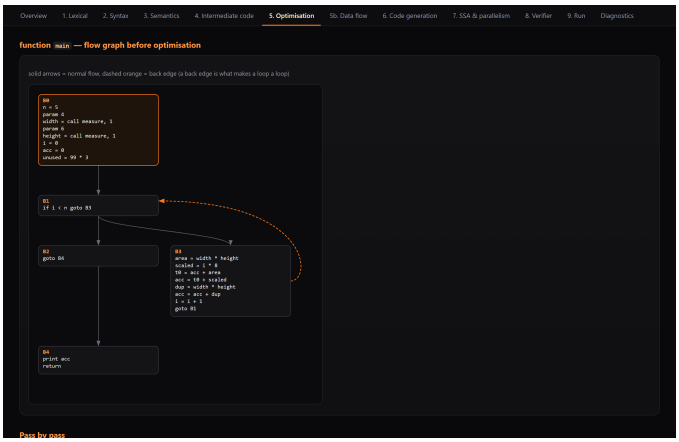
[52] R. Wahbe, S. Lucco, T. E. Anderson, and S. L. Graham, "Efficient software-based fault isolation," in *Proc. 14th ACM Symp. Operating Syst. Princ. (SOSP)*, 1993, pp. 203–216.

[53] X. Leroy, "Formal verification of a realistic compiler," *Commun. ACM*, vol. 52, no. 7, pp. 107–115, Jul. 2009.

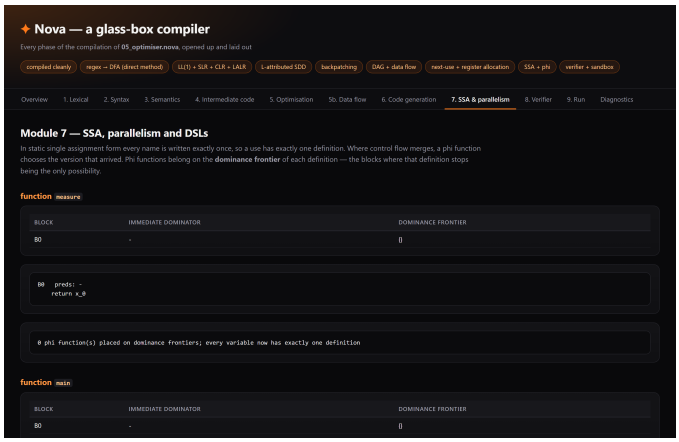
[54] A. Pnueli, M. Siegel, and E. Singerman, "Translation validation," in *Proc. Int. Conf. Tools Algorithms Construction Anal. Syst. (TACAS)*,



(a) Overview: the headline numbers and the source program



(b) Optimisation: the flow graph of `main` before optimisation



(c) SSA and parallelism: dominators and the SSA form of each function

Fig. 50. The self-contained HTML report for `05_optimiser.nova`, which embeds every phase and its flow graphs in one file with no external resources.

LNCS, vol. 1384, 1998, pp. 151–166.

[55] W. M. McKeeman, "Differential testing for software," *Digit. Tech. J.*, vol. 10, no. 1, pp. 100–107, 1998.

[56] P. J. Guo, "Online Python Tutor: Embeddable web-based program visualization for CS education," in *Proc. 44th ACM Tech. Symp. Comput. Sci. Educ. (SIGCSE)*, 2013, pp. 579–584.

[57] E. M. Reingold and J. S. Tilford, "Tidier drawings of trees," *IEEE Trans. Softw. Eng.*, vol. SE-7, no. 2, pp. 223–228, Mar. 1981.

[58] K. Sugiyama, S. Tagawa, and M. Toda, "Methods for visual understanding of hierarchical system structures," *IEEE Trans. Syst., Man, Cybern.*, vol. SMC-11, no. 2, pp. 109–125, Feb. 1981.

[59] A. Aiken, "Cool: A portable project for teaching compiler construction," *SIGPLAN Not.*, vol. 31, no. 7, pp. 19–24, Jul. 1996.

[60] R. Padhye, K. Sen, and P. N. Hilfinger, "ChocoPy: A programming language for compilers courses," in *Proc. ACM SIGPLAN Symp. SPLASH-E*, 2019, pp. 41–45.

[61] S. H. Rodger and T. W. Finley, *JFLAP: An Interactive Formal Languages and Automata Package*. Sudbury, MA, USA: Jones and Bartlett, 2006.

[62] M. Godbolt, "Optimizations in C++ compilers," *Commun. ACM*, vol. 63, no. 2, pp. 41–49, Feb. 2020.

[63] Python Software Foundation, "`http.server`—HTTP servers," *Python 3 Documentation*. [Online]. Available: https://docs.python.org/3/library/http.server.html

[64] WHATWG, "Fetch standard: CORS protocol," Living Standard. [Online]. Available: https://fetch.spec.whatwg.org/

[65] C. Jackson, A. Barth, A. Bortz, W. Shao, and D. Boneh, "Protecting browsers from DNS rebinding attacks," in *Proc. 14th ACM Conf. Comput. Commun. Secur. (CCS)*, 2007, pp. 421–431.

[66] W3C, "Content Security Policy Level 3," W3C Working Draft. [Online]. Available: https://www.w3.org/TR/CSP3/

[67] R. B. Miller, "Response time in man-computer conversational transactions," in *Proc. AFIPS Fall Joint Comput. Conf.*, vol. 33, 1968, pp. 267–277.